Agentic AI 실전 개발

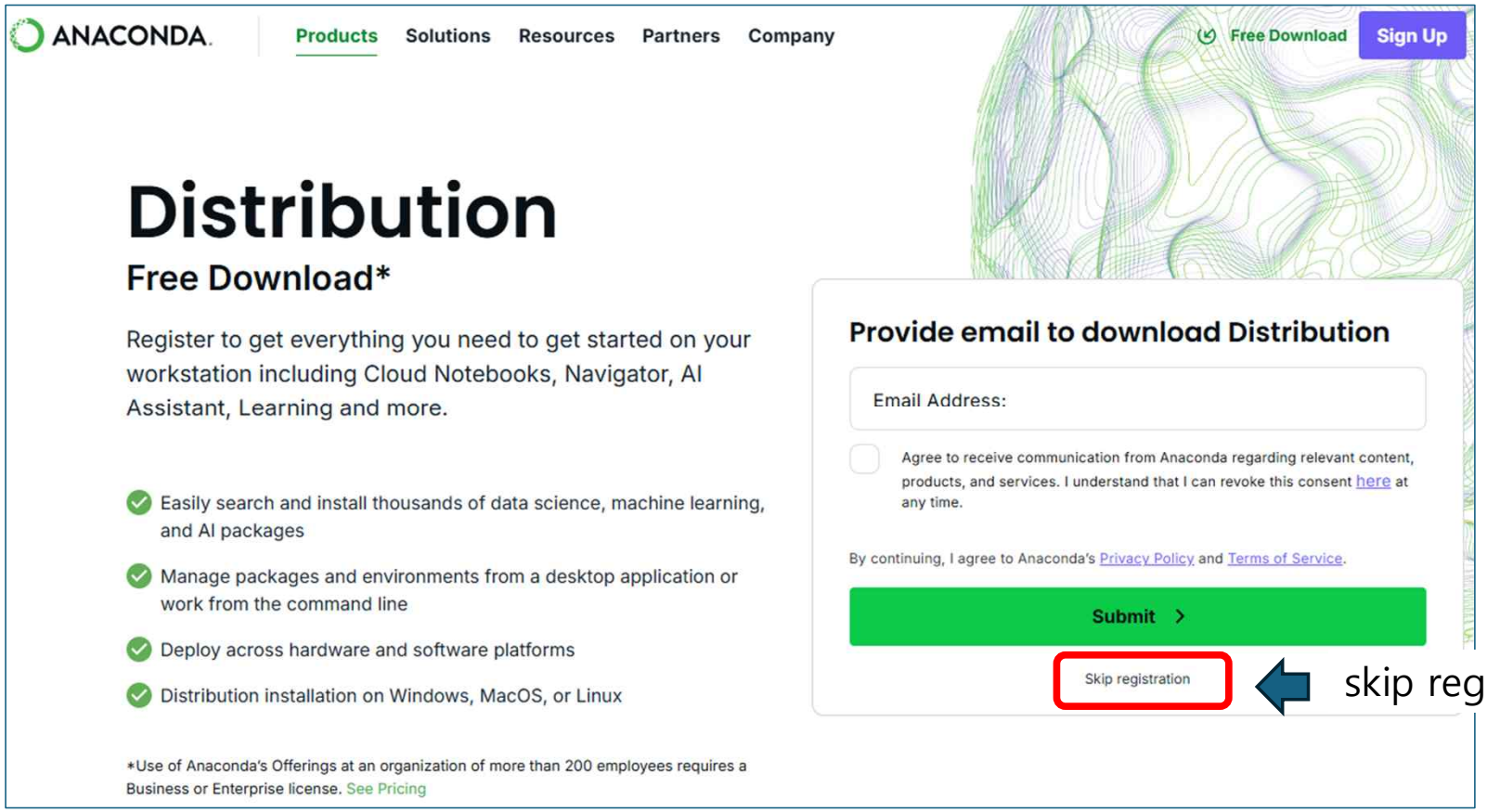
OpenAI Agents SDK · MCP · LangChain

강사 : 오영제

실습 환경 설정

Anaconda 설치

Download from https://www.anaconda.com/download



가상환경 생성 및 package 설치

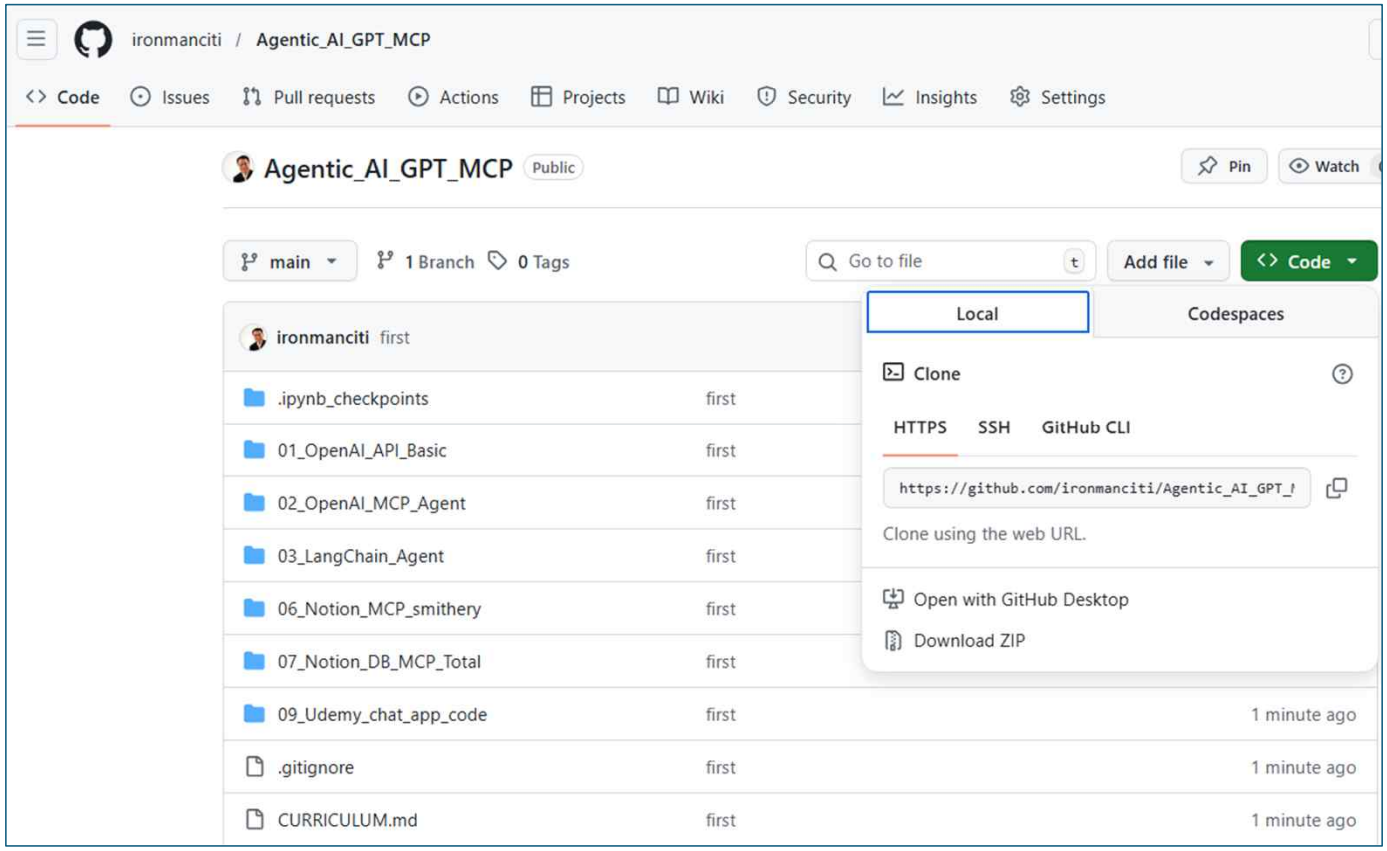
```
conda create -n langchain python=3.12 -y
conda activate langchain
pip install -r requirements.txt
```

- package Install (수동 설치 경우)

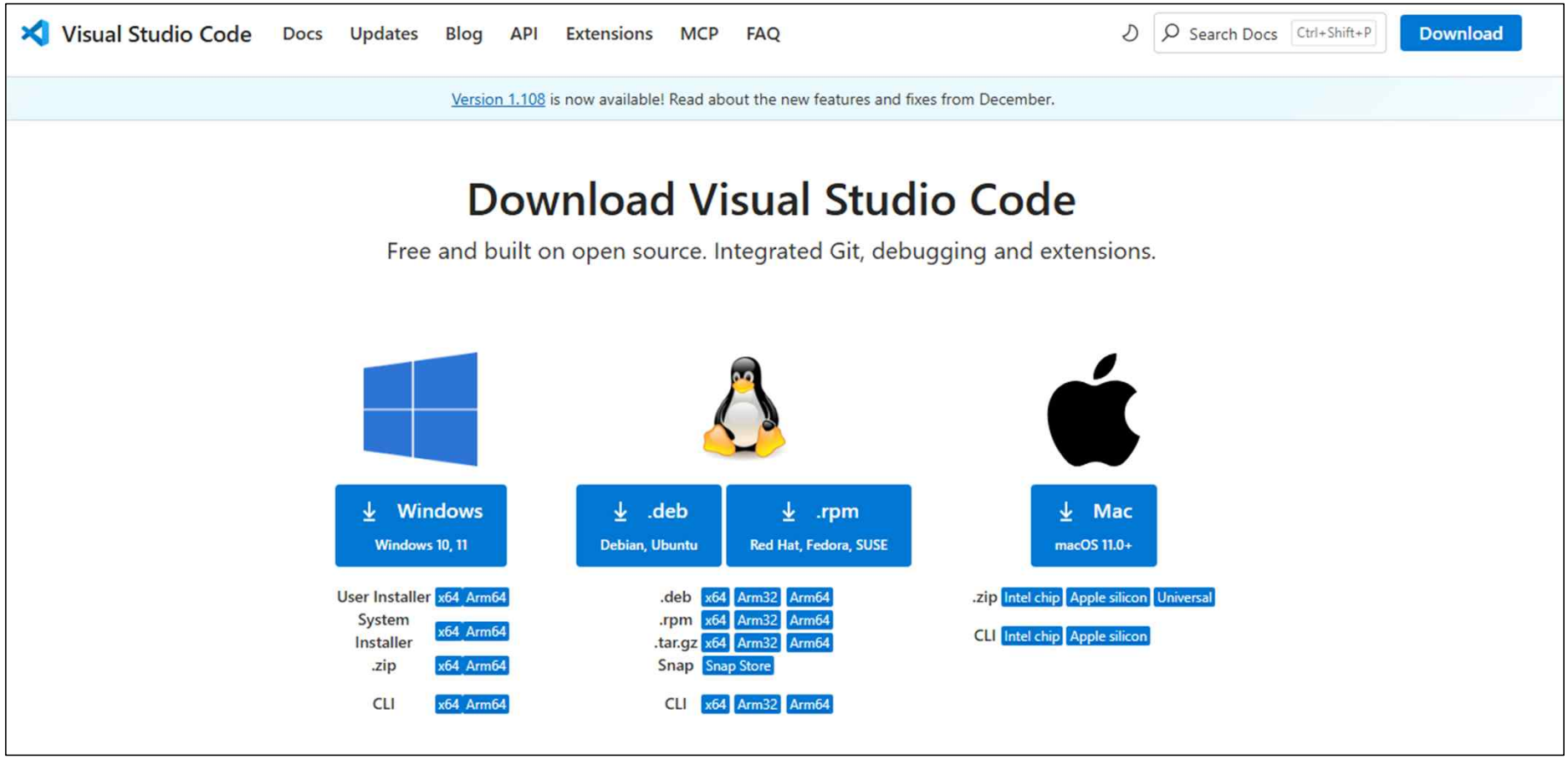
```
pip install -U langchain
pip install -U langchain-openai
pip install openai-agents
```

Github Repository 에서 실습 code download

- github.com/ironmanciti/Agentic_AI_GPT_MCP

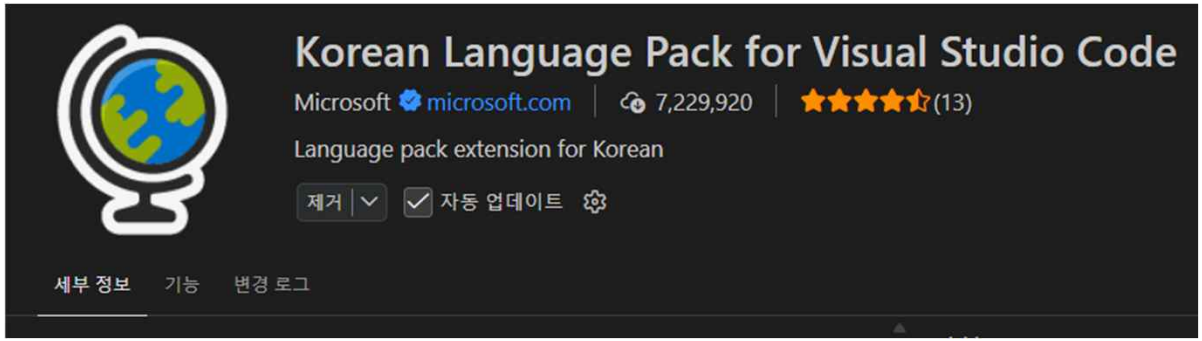


vscode 설치 (https://code.visualstudio.com/download)

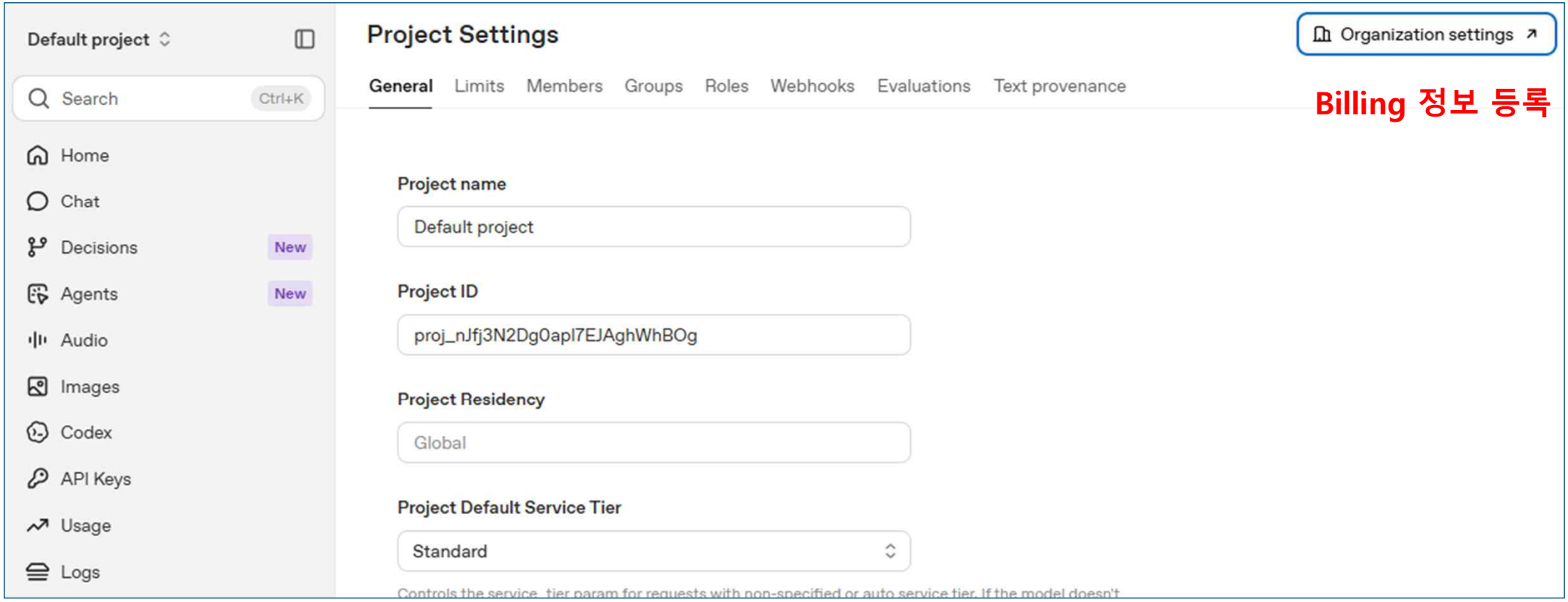


확장팩 설치

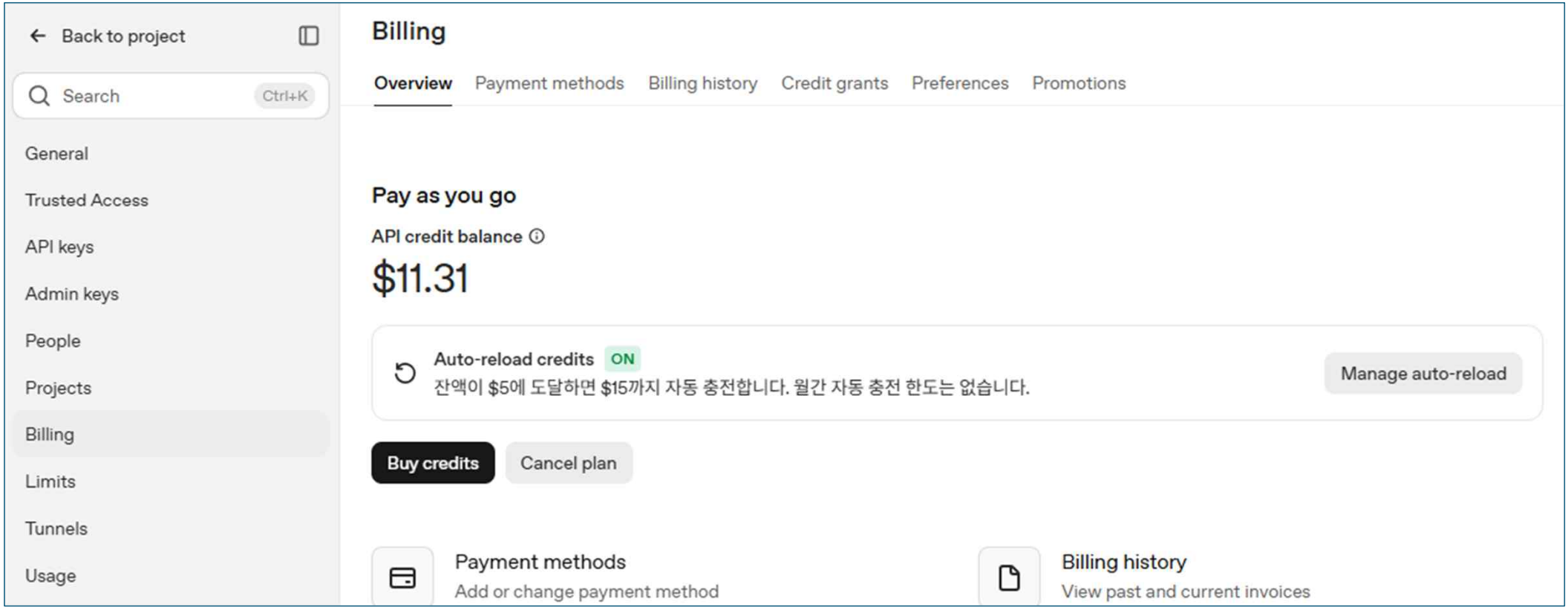

Python
Microsoft microsoft.com
243,329,372
(632)
Python language support with extension access points for IntelliSense (Pylance), Debugging (Python Debugger), linting, formatting, refactoring, unit tests, and more.
시험판 버전으로 전환
사용 안 함
제거
자동 업데이트


Korean Language Pack for Visual Studio Code
Microsoft microsoft.com
7,229,920
(13)
Language pack extension for Korean
제거
자동 업데이트
세부 정보
기능
변경 로그

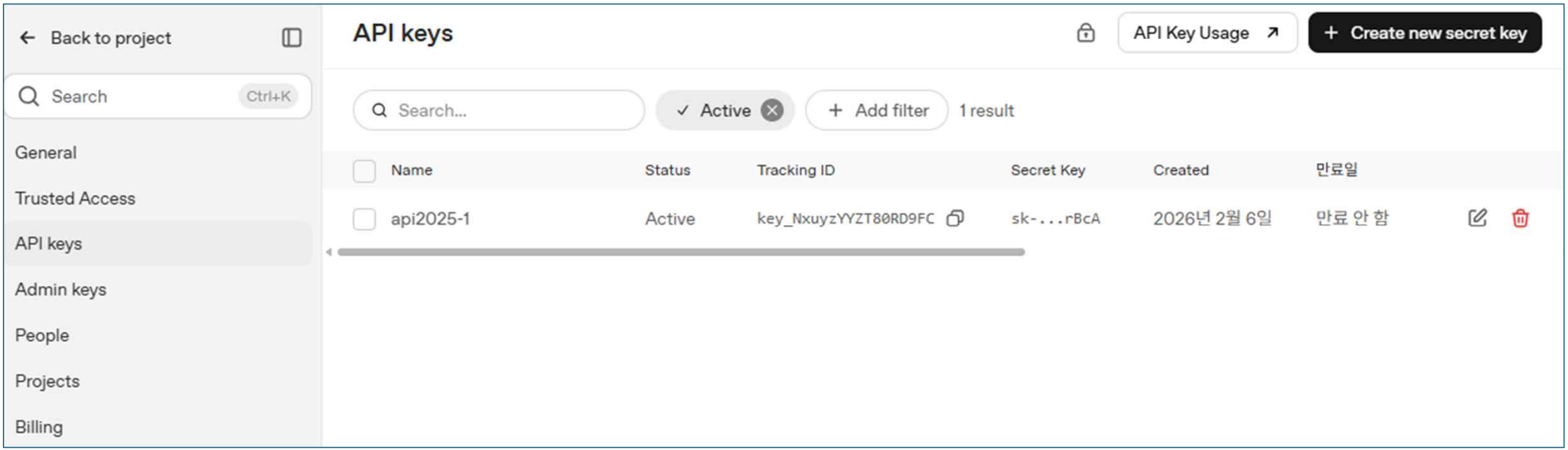
OpenAI API Key 생성 : https://openai.com/index/openai-api/



Billing 정보 등록



API key 생성

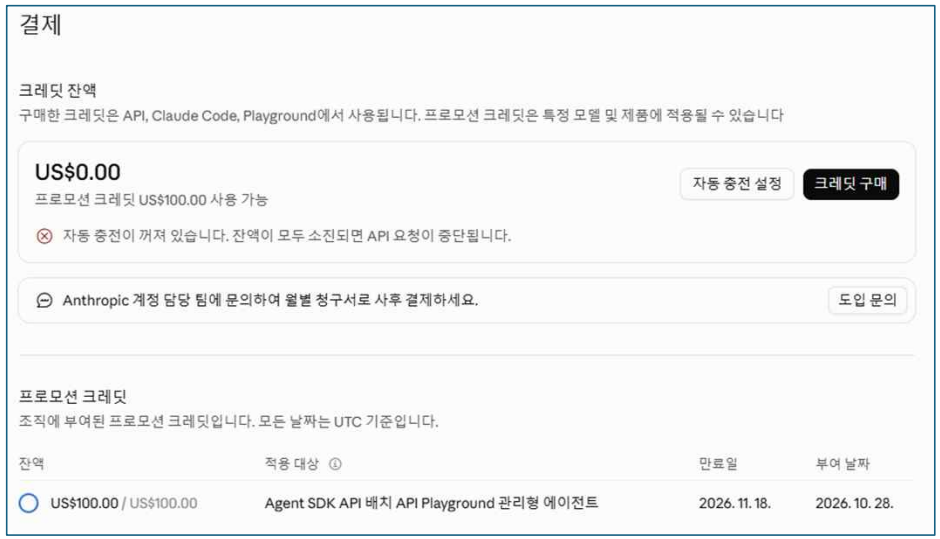

Back to project
Search
Ctrl+K
General
Trusted Access
API keys
Admin keys
People
Projects
Billing
API keys
API Key Usage
Create new secret key
Search...
Active
Add filter
1 result
Name
Status
Tracking ID
Secret Key
Created
만료일
api2025-1
Active
key_NxuyzYYZT80RD9FC
sk-...rBcA
2026년 2월 6일
만료 안 함

Gemini API KEY 생성



Anthropic API KEY 생성





프로모션 크레딧

단일 프로젝트에 대해 API key 설정 방법

- .env – API 키가 포함된 로컬 파일 생성

```
# .env 파일
OPENAI_API_KEY=sk-vvDtll**********XiilEpdjLhBJaH0f
```

- .gitignore 에 .env 파일 포함

```
# .env 파일을 git에서 무시
.env
```

- Python code

```
pip install python-dotenv

from dotenv import load_dotenv
_ = load_dotenv()     # local .env file을 읽어서 os.environ 에 OPENAI_API_KEY  추가

from openai import OpenAI
client = OpenAI()     # os.environ.get("OPENAI_API_KEY")을 default 로 사용하여 API Key 이용
```

OpenAI Agents SDK

정적인 시스템에서 지능형 시스템으로

Chatbot
정적 문답
→
RAG App
지식 검색 결합
→
RAG + MCP
외부 도구 연결
→
Agentic AI
자율 판단·실행

LLM 단독으로는 불가능

- 실시간 데이터 접근
- 파일 열기·조작
- 개인·사내 문서 이해

해결: 도구(Tools)를 에이전트 루프로 호출 → 아래 툴킷으로 구현

툴킷	위치	특징
LangGraph (LangChain)	프로덕션 최다 사용	그래프로 상태·흐름 세밀 제어
OpenAI Agents SDK	빠르게 성장	가볍고 단순, 핸드오프·가드레일 내장
Claude Agent SDK	빠르게 성장	자율형 작업(코딩·파일)에 강함
Google ADK	Google Cloud 생태계	Gemini, Vertex AI 연동
CrewAI	프로토타이핑	역할 기반 멀티 에이전트
Microsoft Agent Framework	기업, .NET 환경	AutoGen + Semantic Kernel
Pydantic AI / LlamaIndex	특화 영역	타입 안정성 / RAG 중심

MCP가 도구 연결의 표준 · 모델사 SDK vs 모델 중립 프레임워크로 양분

2026년 중반 기준 정리 · 점유율 통계가 아닌 일반적 흐름 · 파란 행 = 교재에서 다루는 툴킷

Agents: ReAct 루프의 핵심

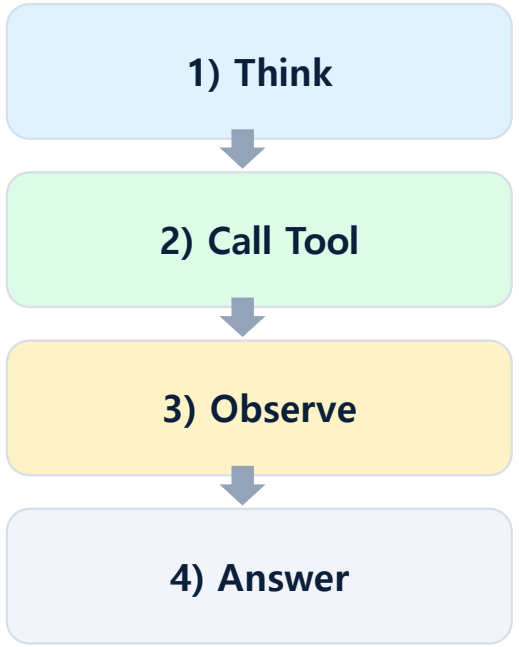
Agent 란 ?

- LLM을 핵심 엔진으로 사용하여 주어진 목표를 달성하기 위해
  독립적으로 작업(추론, 판단, 실행, 피드백)을 수행하는 인공지능 시스템

- 주로 LLM(대규모 언어 모델)의 능력을 기반으로 동작하며,
  사용자의 명령을 이해하고, 판단하며, 실행

- 즉, LLM은 두뇌, Agent는 이 두뇌를 활용해 행동을 실행하는 작업자 역할

- 활용 사례
  - 고객 지원 챗봇
  - 연구 도우미
  - 개인 비서
  - 데이터 분석 에이전트
  - 교육용 튜터



OpenAI Agents 구성 요소

OpenAI Agents SDK 핵심 구성 요소

- **모델 (Model)** : 고급 추론 능력과 에이전트 기능 내재 모델
- **에이전트 (Agents)** : LLM에 지시문과 도구를 장착한 실행 단위
- **핸드오프 (Handoffs)** : 특정 작업을 다른 에이전트에게 위임할 수 있도록 해 줌
- **가드레일 (Guardrails)** : 에이전트에게 전달되는 입력을 검증하는 장치
- **함수 기반 도구 (Function Tools)** : 에이전트가 호출할 수 있는 Python 함수 기반 도구
- **세션 (Session)** : 에이전트 실행 반복 과정(Agent Loop) 동안 필요한 작업 맥락을 유지해주는 영속적 메모리 레이어
- **추적 (Tracing)** : 에이전트 실행에 대한 내장 추적 기능

Structured Output: “JSON으로 고정”하기

구조화된 출력 (Structured Output)

- 모델이 특정한 출력 구조를 따르도록 지시 받는 것
- 모델의 응답이 데이터베이스 스키마나 다른 정해진 형식에 부합하도록 보장
  → 모델 출력의 일관성 유지
- 구조화된 출력 구현 방법 → 출력 구조를 스키마로 정의
- 스키마는 JSON과 같은 단순한 형식으로 표현
- Pydantic과 같은 도구를 사용하여 타입 힌트와 유효성 검사를 포함한 스키마 정의

```python
from pydantic import BaseModel
from agents import Agent, Runner

# 캘린더 이벤트 데이터 구조 정의
class CalendarEvent(BaseModel):
    name: str              # 이벤트 이름
    date: str              # 이벤트 날짜
    participants: list[str]  # 참가자 목록

# 에이전트 정의: 텍스트에서 캘린더 이벤트 정보를 추출
agent = Agent(
    name="캘린더 추출기",  # 에이전트 이름
    instructions="텍스트에서 캘린더 이벤트를 추출하세요. "
                 "이벤트 이름, 날짜, 참가자 정보를 구조화된 데이터로 반환하세요.",
    output_type=CalendarEvent,  # 반환 데이터 형식
    model="gpt-5-mini",            # 사용할 LLM 모델
)
```

Tools & Tool Calling

LLM의 함수 호출 방법

LLM은 함수 호출(Function Calling) 기능을 통해 외부 도구 및 API와 안정적으로 통합될 수 있다.

JSON 기반 함수 호출:
- 모델은 함수 시그니처를 따르는 JSON 객체로 응답
- 개발자는 함수의 입력 파라미터를 JSON 스키마로 설명

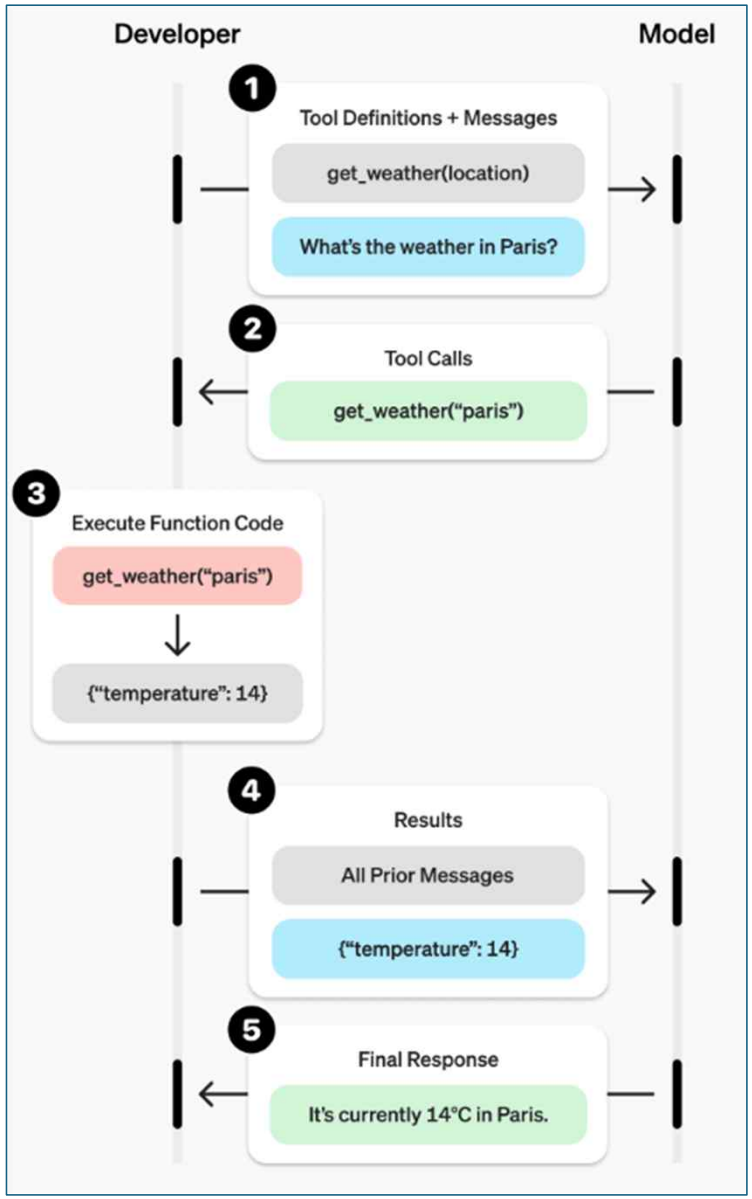
자동 함수 호출 결정:
- 모델은 입력에 따라 적절한 함수를 호출할 시점을 감지

사용 사례:
- 외부 도구 호출: API, 플러그인 등과 연결해 작업 수행.

```python
@function_tool
def multiply(x: float, y: float) -> float:
    """
    x 와 y 를 곱한다.
    """
    print("** multiply 함수 실행 **", x, y)
    return x * y

@function_tool
def get_weather(위도: float, 경도: float) -> str:
    print(위도, 경도)
    print(f"Weather 함수 실행 - 도시: {위도, 경도}")
    latitude=위도
    longitude=경도
    response = requests.get(f"https://api.open-meteo.com/v1/forecast?latitude={l
    data = response.json()
    return data['current']['temperature_2m']

agent = Agent(
    name="Assistant",
    instructions=(
        "당신은 유용한 도우미 입니다. 가능하면 제공된 도구를 사용하세요."
        "당신은 기상 전문가 입니다. 도시의 경도와 위도를 주면 날씨를 알려줍니다."
        "사용자 질의에 답하기 위해 당신의 지식에만 너무 의존하지 말고 도구를 사용하세요."
    ),
    model="gpt-5-nano",
    tools=[multiply, get_weather]
)
```



- 함수 호출 단계

(1) 함수 정보와 함께 LLM 모델 호출

(2) 모델이 함수 호출을 결정 – 함수명과 argument 반환

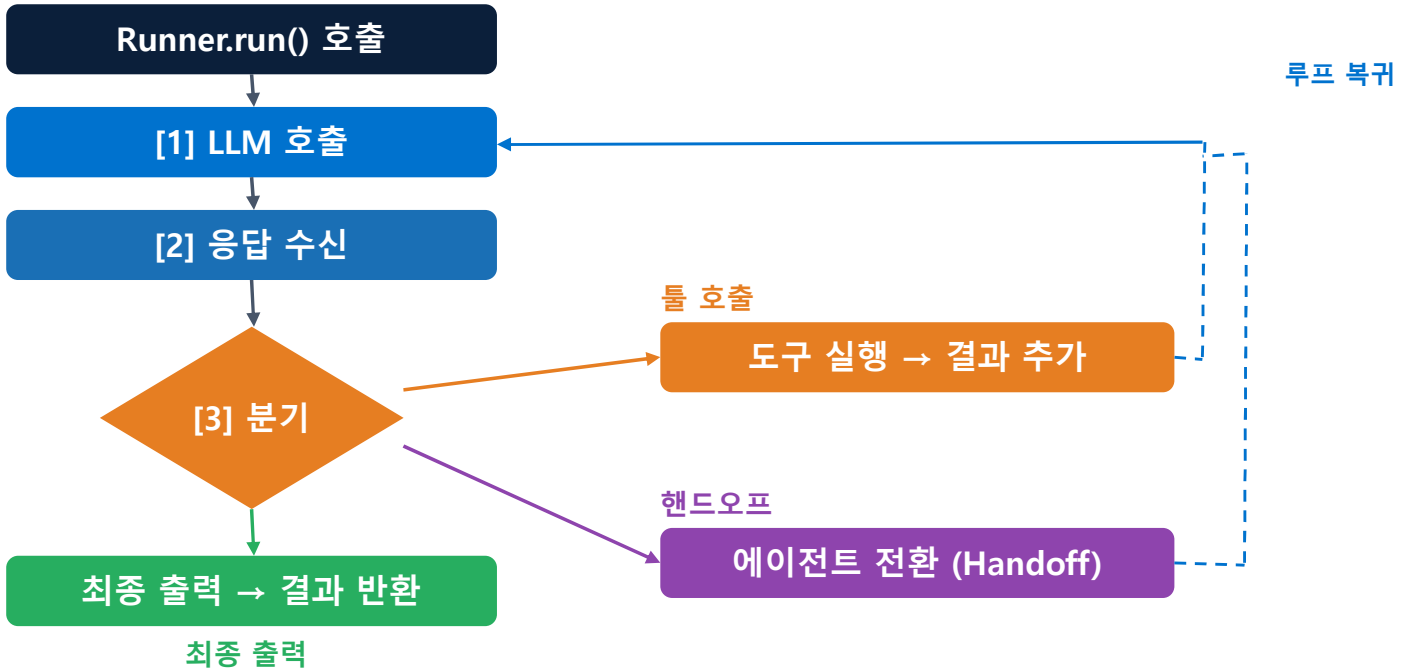
(3) 함수 호출 – 모델이 만들어준 argument 이용

(4) 함수 호출 결과를 모델에 제공

(5) 모델이 최종 응답 생성

Agent Loop 흐름

에이전트 루프 동작 순서



툴 호출/핸드오프 → [1]로 돌아가 LLM 재호출, 최종 출력 시 루프 종료

Start [1] LLM [2] 응답 툴 호출 핸드오프 완료

output_type 3가지 케이스

output_type에 따른 최종 출력 결정

Case 1: output_type 미지정

- LLM이 자연어 문자열로 직접 응답 → result.final_output = 문자열

Case 2: output_type 지정 (구조화된 출력)

- Pydantic 모델 지정 → LLM 응답이 모델에 맞으면 구조화된 객체 반환

```
class WeatherResult(BaseModel):
    city: str; temperature: float; condition: str
agent = Agent(output_type=WeatherResult, tools=[get_weather])
```

Case 3: 핸드오프 + output_type

- 분류(Triage) 에이전트 → 전문 에이전트로 핸드오프
- 전문 에이전트에 output_type 설정 → 구조화된 결과 반환

핸드오프 (Handoffs)

핸드오프 (Handoffs) 란?

- 에이전트가 **다른 에이전트에게 작업을 위임**하는 기능
- 각 에이전트가 **서로 다른 전문 영역**을 담당할 때 유용

  예) 고객 지원 앱에서 **주문 조회**, **환불**, **FAQ** 등을 각각 전담하는 에이전트 구성

핵심 동작 원리

- Handoff는 LLM에게 **도구(Tool)**로 표현됨

  예) "Refund Agent"로의 핸드오프 → LLM이 **transfer_to_refund_agent** 도구 호출

핸드오프 (Handoffs) 구현 방법

Handoff 생성 방법

방법 1: handoffs 파라미터에 Agent 객체 직접 전달

- handoffs 파라미터에 위임 대상 **Agent** 객체를 전달
- **handoff_description** 설정 → LLM이 **언제 이 핸드오프를 선택할지** 판단하는 힌트

방법 2: handoff() 함수 사용

- 위임 대상 에이전트를 지정하면서 **세부 제어**가 가능
- 추가 설정(**override**)이나 **입력 필터링** 같은 고급 기능 활용 가능

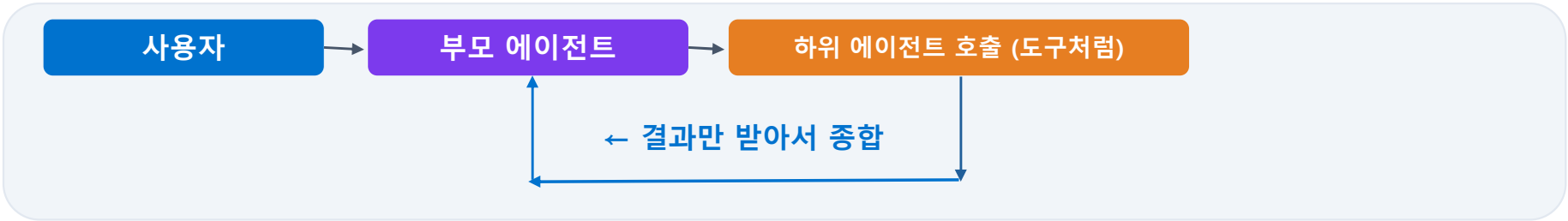
agent.as_tool() vs Handoff

Handoff - 제어권 완전 이전



예: 고객센터에서 환불 담당자에게 연결해주는 것

agent.as_tool() - 부모가 제어권 유지



예: 부모가 요약도 시키고 번역도 시킨 뒤 결과를 모아서 사용자에게 답변

실습 : 01_Agent_Overview

01. OpenAI Agent (Agents Overview)

OpenAI 플랫폼을 사용하여 사용자를 대신해 행동(예: 컴퓨터 제어 등)을 수행할 수 있는 **에이전트(Agent)** 를 구축할 수 있습니다.
Python용 **Agent SDK**를 사용하면 백엔드에서 이러한 에이전트의 **오케스트레이션(조율) 로직**을 만들 수 있습니다.

OpenAI Agents SDK는 **에이전트 기반 AI 애플리케이션**을 개발할 수 있게 해주는 도구입니다.

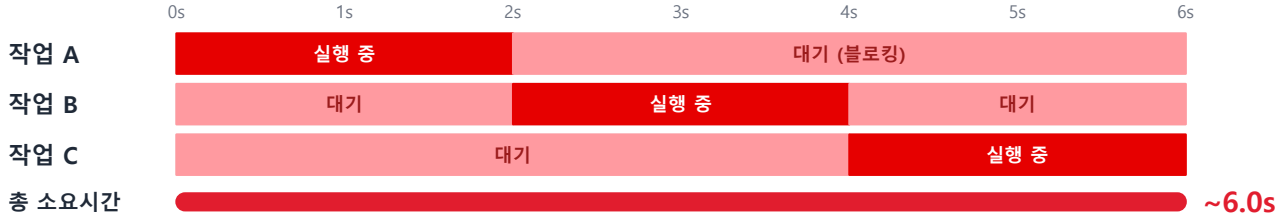
주요 기능 요약:

- **에이전트 루프 (Agent Loop)**:
  도구 실행 → 결과 전달 → LLM 호출 → 반복 실행 → 완료까지 자동 처리
- **핸드오프 (Handoffs)**:
  여러 에이전트 간의 **협업과 위임**을 유연하게 처리 가능
- **가드레일 (Guardrails)**:
  에이전트 입력을 **사전 검사/검증**하여, 조건을 만족하지 않으면 **조기 종료 가능**
- **함수 기반 도구 (Function Tools)**:
  Python 함수 하나를 **자동으로 에이전트 도구로 변환**,
  Pydantic 기반 스키마 자동 생성 및 검증 포함

Runner 3종 비교 — 동기 vs 비동기 실행

메서드	호출 방식	특징	주요 사용처
`Runner.run()`	await (비동기)	에이전트 루프 자동 실행 · I/O 대기 중 다른 작업 처리	Jupyter · FastAPI · 웹앱
`Runner.run_sync()`	동기 호출 (await 불필요)	내부에서 이벤트 루프 실행 · 끝날 때까지 블로킹	.py 스크립트 · 테스트
`Runner.run_streamed()`	await 없이 호출 + async for	토큰 · 이벤트 단위 실시간 출력	채팅 UI

SYNC **동기 실행 (run_sync) — 순차, 블로킹**



ASYNC **비동기 실행 (run) — 동시, 논블로킹**



LLM 호출 = I/O 대기의 연속

동기는 응답을 기다리는 동안 아무것도 못 하지만, 비동기는 그 사이 다른 작업을 처리한다 → 여러 에이전트·도구 호출이 겹칠수록 차이가 커진다

주의

run_streamed()는 코루틴이 아니므로 호출에 await를 붙이지 않고, 반환된 RunResultStreaming의 stream_events()를 async for로 순회

Jupyter는 이미 이벤트 루프가 돌고 있어 run_sync() 대신 await Runner.run() 사용

동기 vs 비동기 — 왜 Runner 실행 방법이 두 가지인가

실행 엔진은 비동기 하나 — `Runner.run()` · run_sync는 그 엔진을 동기 코드에서 부를 수 있게 감싼 래퍼

`Runner.run_sync(agent, q)` → 이벤트 루프를 새로 만들어 Runner.run() 실행 → 끝날 때까지 기다림 → result (동기 반환)

`await Runner.run()` 이벤트 루프가 이미 돌고 있는 곳

어디서 쓰나

- Jupyter 노트북 — 01~10 실습 전부
- FastAPI 등 async 웹 서버
- async 함수 안 — 예: mcp_client.py의 run_agent()

왜 이걸 쓰나

- 여러 에이전트 동시 실행: asyncio.gather → 3개 순차 6초를 약 2초로
- 스트리밍(run_streamed) · 가드레일 · 여러 tool call 병렬 처리
- 기다리는 동안 다른 요청을 막지 않음 (서버 동시 처리)

주의 await는 async 함수나 Jupyter 셀 안에서만 쓸 수 있다

`Runner.run_sync()` 이벤트 루프가 없는 곳

어디서 쓰나

- Flask 요청 처리 — 11_rag-chat-flask/app.py
- 일반 .py 스크립트 · CLI · 배치 작업
- pytest 같은 동기 테스트 코드

왜 이걸 쓰나

- await 없이 한 줄로 결과를 받는다 — 코드가 단순
- 에이전트를 한 번만 실행하면 속도 차이가 없다
- 동기 프레임워크(Flask)에 그대로 끼워 넣을 수 있다

주의 이미 루프가 도는 Jupyter · FastAPI에서 쓰면 에러

선택 기준 — 지금 이벤트 루프가 돌고 있나? 예 → `await Runner.run()` 아니오 → `Runner.run_sync()`

Flask라도 MCP 서버 연결(async with)처럼 비동기 작업이 섞이면 async 함수로 묶어 asyncio.run()으로 실행 — 02_OpenAI_MCP_Agent/mcp_client.py 방식

실습 : 02_Agent_Runner

02. Agent와 Runner

Runner는 에이전트 루프를 돌리는 실행기입니다.

- **동기**: `Runner.run_sync(agent, input)` — 스크립트/테스트에 적합
- **비동기**: `await Runner.run(agent, input)` — Jupyter/비동기 앱에 적합

에이전트 루프 동작:

1. LLM 호출 (에이전트 설정 + 메시지 히스토리)
2. 응답에 툴 호출이 있으면 실행 후 결과를 메시지에 추가하고 1번으로
3. 핸드오프가 있으면 대상 에이전트로 바꾸고 1번으로
4. **final_output**이 나오면 루프 종료 후 반환
5. `max_turns`로 최대 턴 수 제한 가능

Error Handling — max_turns 초과와 error_handlers

기본값 max_turns=10 | 1턴 = LLM 호출 1회 | 변경: Runner.run(..., max_turns=20) | 너무 크면 무한 루프 비용 증가

핸들러 없음

max_turns 초과
(예: max_turns=2)
→ **MaxTurnsExceeded**
예외 발생
→ **프로그램 중단**
try/except 없으면 에러 화면

핸들러 등록

max_turns 초과
(같은 코드)
→ **핸들러 실행**
RunErrorHandlerResult 반환
→ **폴백 메시지 반환**
result.final_output

`RunErrorHandlerResult` `final_output` 예외 대신 돌려줄 폴백 메시지 `include_in_history=False` 대화 기록에 남기지 않음

핸들러는 def · async def 모두 가능 — 무엇이 다른가: 턴 초과 시 핸들러가 5개 도시 날씨를 조회하는 동안, 다른 코루틴이 0.5초마다 한 줄씩 출력

def + requests (동기)

```
[다른 코루틴] 실행 중
>> [핸들러] 시작
>> 서울 25.9°C
>> 도쿄 25.1°C
>> 뉴욕 31.8°C
>> 런던 · 파리 ...
>> [핸들러] 완료
[다른 코루틴] 실행 중
```

시작~완료 사이 **[다른 코루틴] 없음**
→ 이벤트 루프 블로킹

async def + httpx (비동기)

```
>> [핸들러] 시작
[다른 코루틴] 실행 중
>> 서울 25.9°C
[다른 코루틴] 실행 중
>> 도쿄 25.1°C
>> 뉴욕 31.8°C
[다른 코루틴] 실행 중
>> 런던 · 파리 ...
>> [핸들러] 완료
```

시작~완료 사이에도 **[다른 코루틴] 계속 출력**
→ await마다 제어권 양보

구분	def	async def
이벤트 루프	블로킹	await 시 양보
다른 코루틴	대기	동시 실행
적합한 작업	단순 문자열 반환	DB · HTTP · 로깅 등 I/O

실무 결론
웹 서버에서 동기 핸들러의 블로킹 = 다른 모든 사용자 요청이 멈추는 시간. I/O가 있으면 async def + 비동기 클라이언트 (httpx)

실습 : 03_error_handling

03. Error Handling

에이전트 루프에서 **max_turns**를 초과하면 기본적으로 `MaxTurnsExceeded` 예외가 발생하여 프로그램이 중단됩니다.
error_handlers를 사용하면 예외 대신 **제어된 최종 메시지**를 반환하도록 할 수 있습니다.

핵심 개념

- `error_handlers={"max_turns": handler}` : `Runner.run()` 호출 시 전달하여, 턴 초과 시 핸들러 함수가 실행되도록 설정
- `RunErrorHandlerResult` : 핸들러가 반환하는 결과 객체로, `final_output` (폴백 메시지)과 `include_in_history` 옵션을 포함
- `include_in_history=False` : 폴백 메시지를 대화 기록에 남기지 않음 (사용자 안내 전용)
- **동기(`def`) vs 비동기(`async def`) 핸들러**: 동기 핸들러는 실행 중 이벤트 루프를 블로킹하고, 비동기 핸들러는 `await` 시 제어권을 양보 → I/O 작업에는 비동기 권장

학습 내용

1. `max_turns` 초과 시 `error_handlers`로 예외 없이 안전하게 처리하기
2. 비동기(`async def`) / 동기(`def`) 핸들러 작성법
3. `include_in_history` 옵션의 역할과 활용
4. 동기 vs 비동기 핸들러의 **이벤트 루프 블로킹 차이** 관찰 실험 (틱 간격 측정)

on_handoff 콜백 — 개념

핸드오프 발생 시 자동 실행되는 콜백

- **실행 시점**
  - LLM이 핸드오프 도구(transfer_to_<agent>)를 호출하는 순간, 제어권 이전 직전에 실행
- **용도 — 사이드 이펙트 처리**
  - 로깅, 알림 발송, 상태 업데이트, 다음 에이전트를 위한 데이터 준비



콜백은 ①과 ② 사이에서 실행 — 대상 에이전트가 응답을 만들기 전

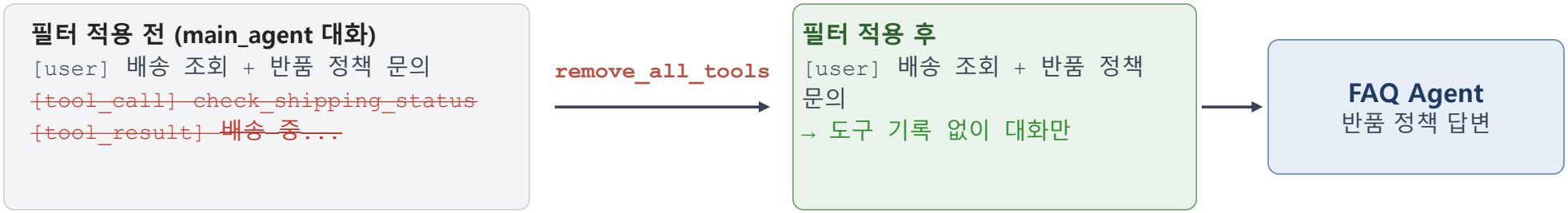
핸드오프는 LLM에게 transfer_to_<agent_name> 이름의 도구로 노출되며, on_handoff는 그 도구가 호출되는 순간의 훅(hook)이다.

input_filter — 대화 기록 필터링

다음 에이전트에 전달되는 히스토리 제어

기본 동작: 핸드오프 시 새 에이전트는 이전 대화 기록 전체(도구 호출/결과 포함)를 그대로 이어받음 → input_filter로 가공 가능

내장 필터 (agents.extensions.handoff_filters)	동작	사용 예
`remove_all_tools`	도구 호출/결과 메시지를 모두 제거	FAQ 에이전트에 깔끔한 대화만 전달
`nest_handoff_history`	이전 대화 전체를 단일 assistant 메시지로 압축	요약 에이전트에 컨텍스트만 전달




```
handoff(faq_agent, input_filter=handoff_filters.remove_all_tools)    # 함수 참조 전달(괄호 없음)
```

실습 : 04_Handoffs

04. Handoffs (핸드오프 심화)

핸드오프(Handoff) 는 에이전트가 특정 작업을 다른 에이전트에게 위임하는 기능입니다.
핸드오프는 LLM에게 **도구(tool)** 로 표현됩니다.
예) `Refund Agent` 로의 핸드오프 → LLM 도구 이름: `transfer_to_refund_agent`

방식	예시	특징
Agent 직접 전달	`handoffs=[billing_agent]`	간단, 기본 동작
`handoff()` 함수	`handoffs=[handoff(billing_agent, on_handoff=...)]`	콜백, 이름 재정의, 입력 필터 등 고급 옵션

`handoff()` 함수의 주요 파라미터:

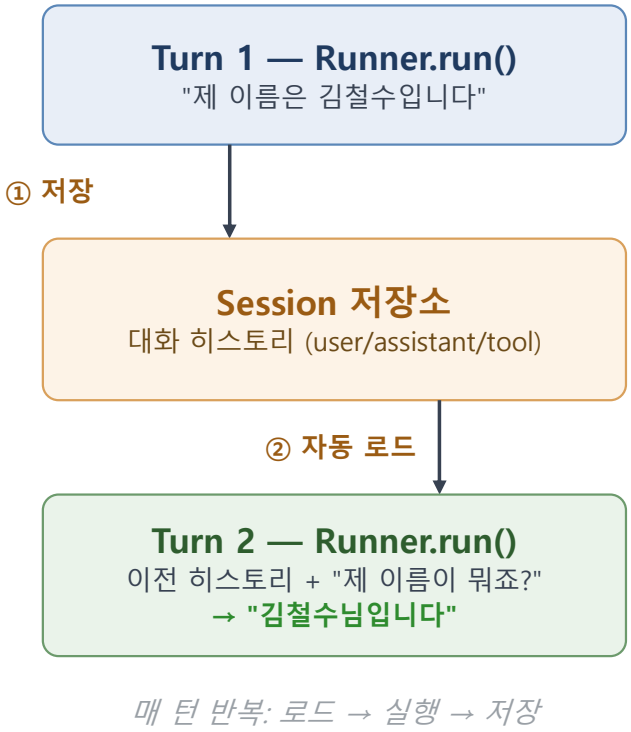
파라미터	설명
`agent`	위임할 대상 에이전트
`tool_name_override`	LLM에게 노출되는 도구 이름 재정의 (기본: `transfer_to_<agent_name>`)
`tool_description_override`	도구 설명 재정의
`on_handoff`	핸드오프 호출 시 실행되는 콜백 함수
`input_type`	LLM이 핸드오프 시 함께 전달할 데이터 타입 (Pydantic 모델)
`input_filter`	다음 에이전트에게 전달되는 대화 기록 필터링 함수
`is_enabled`	핸드오프 활성화 여부 (bool 또는 런타임 함수)

Session — 개념과 필요성

멀티턴 대화 기록의 자동 관리

- **문제 — LLM 호출은 상태 비저장(stateless)**
  - Runner.run()은 매번 독립 실행 → 이전 턴의 대화를 기억하지 못함
  - 수동 해법: result.to_input_list()를 다음 입력에 직접 이어붙이기
    → 코드 번거로움, 누락·중복 위험

- **해결 — Session: 대화 기록 저장소**
  - 실행 전: 저장된 히스토리를 자동 로드해 입력 앞에 결합
  - 실행 후: 이번 턴의 user/assistant/tool 아이템을 자동 저장
  - 같은 session_id로 실행하면 같은 대화를 계속 이어감




```python
session = SQLiteSession("user_123")
await Runner.run(agent, "제 이름은 김철수입니다", session=session)
await Runner.run(agent, "제 이름이 뭐죠?", session=session)
# → "김철수님입니다" — 이전 턴을 기억
```

핵심: Session은 Runner 바깥에 있는 대화 메모리 — 어디에 저장하느냐에 따라 다음 슬라이드의 10종 구현으로 나뉜다

Session 유형 비교

SDK 제공 Session 10종

- **SQLiteSession** : 내장, 인메모리/파일 (개발/프로토타입)
- **AsyncSQLiteSession** : 비동기 전용 SQLite
- **AdvancedSQLiteSession** : 브랜치, 토큰 분석 (고급)
- **SQLAlchemySession** : PostgreSQL, MySQL (프로덕션)
- **RedisSession** : Redis 분산 저장 (대규모 서비스)
- **MongoDBSession** : MongoDB 도큐먼트 저장 (NoSQL 스택)
- **DaprSession** : Dapr state store 추상화 — 백엔드 교체 가능 (K8s/클라우드)
- **EncryptedSession** : 암호화 래퍼 (보안 필수)
- **OpenAIConversationsSession** : OpenAI 서버 저장
- **CustomSession** : 사용자 정의 (Django 등)

실습 : 05_Sessions

05. Sessions

Session은 여러 번의 `Runner.run` 호출에 걸쳐 **대화 히스토리를 자동으로 유지**하는 기능입니다.

- 세션을 쓰지 않으면 매 턴마다 이전 메시지 목록을 직접 구성해 넘겨야 합니다.
- `session` 을 넘기면 SDK가 대화를 저장소에 저장/불러와서, 같은 `session` 으로 실행할 때마다 맥락이 이어집니다.

활용: 채팅봇, 고객지원 에이전트 등 다회차 대화.

SDK 제공 Session 유형 (10종) ¶

세션	위치	특징	용도	본 노트북 실습
`SQLiteSession`	`agents`	기본 내장, 인메모리/파일 저장	개발/프로토타입	✅
`AsyncSQLiteSession`	`agents.extensions.memory`	비동기 전용 SQLite	비동기 환경	✅
`AdvancedSQLiteSession`	`agents.extensions.memory`	브랜치, 토큰 사용량 분석	고급 분석 필요 시	✅
`SQLAlchemySession`	`agents.extensions.memory`	PostgreSQL, MySQL 등 지원	프로덕션 DB	✅
`EncryptedSession`	`agents.extensions.memory`	다른 세션을 암호화로 래핑	보안 필요 시	✅
`OpenAIConversationsSession`	`agents`	OpenAI 서버에 대화 저장	OpenAI 플랫폼 활용	✅
`CustomSession`	직접 구현	`Session` 프로토콜 구현	Django, MongoDB 등 연동	✅
`RedisSession`	`agents.extensions.memory`	Redis 기반 분산 저장	대규모 서비스	❌ (Redis 서버 필요)
`MongoDBSession`	`agents.extensions.memory`	MongoDB 도큐먼트 저장	NoSQL 스택	❌ (MongoDB 필요)
`DaprSession`	`agents.extensions.memory`	Dapr state store 추상화	K8s/클라우드	❌ (Dapr 사이드카 필요)

가드레일 (Guardrails) — 개요

에이전트의 입력과 출력을 검증하는 안전장치

에이전트와 함께 실행되며, **입력(Input)**과 **출력(Output)**에 대해 검사·유효성 검증 수행조건 위반 시 **tripwire(즉시 중단 신호)** 발동

구분	Input Guardrail	Output Guardrail
검증 시점	에이전트 실행 전	에이전트 실행 후
검증 대상	사용자 입력	에이전트 출력
데코레이터	`@input_guardrail`	`@output_guardrail`
발동 예외	`InputGuardrailTripwireTriggered`	`OutputGuardrailTripwireTriggered`
실행 모드	병렬(Parallel) 또는 차단(Blocking)	항상 순차(Sequential) — 응답 생성 후 검사

Input Guardrail — 두 가지 구현 방식

공통 구조 — 모든 Input Guardrail은 같은 시그니처의 함수

```python
@input_guardrail
async def guard(ctx, agent, input_data):   # input_data: str 또는 메시지 리스트
  # guardrail 검사 로직 (에이전트 실행 전)
  return GuardrailFunctionOutput(output_info=..., tripwire_triggered=bool)

triage = Agent(..., input_guardrails=[guard])    # 메인 에이전트 실행 전 사용자 입력 검사
```

스타일 1 — 데코레이터 사용

`@input_guardrail` 만 붙이면 함수가 곧바로 `InputGuardrail` 인스턴스가 됨

`@input_guardrail(name=..., run_in_parallel=False)`
→ 데코레이터 인자로 옵션 지정 가능

- **코드가 짧고 간결 — 함수만 리스트에 전달**
- 노트북 예제(homework_guardrail)에서 사용한 방식

스타일 2 — 수동 래핑

`InputGuardrail(guardrail_function=함수)` 로 직접 감싸서 `input_guardrails` 에 전달

- **"가드레일 = 함수 래퍼" 구조가 코드에 명시적**
- 내부적으로 두 방식은 완전히 동일하게 동작

Output Guardrail — 두 가지 구현 방식

공통 구조 — 모든 Output Guardrail은 같은 시그니처의 함수

```python
@output_guardrail
async def guard(ctx, agent, output):
  # guardrail 검사 logic
  return GuardrailFunctionOutput(output_info=..., tripwire_triggered=bool)

assistant = Agent(..., output_guardrails=[guard])    # 응답 생성 후 항상 순차 실행
```

방식 1 — 정규식 기반

`pii_guardrail` — 이메일·전화번호 패턴 탐지

`re.search(email/phone) → has_pii`

`→ tripwire_triggered=has_pii`

- **빠름·결정적·추가 비용 없음**
- 정의한 패턴만 탐지 — 의미는 이해 못 함

방식 2 — LLM 기반

`llm_content_guardrail` — 검증 전용 에이전트 활용

`checker_agent(output_type=GuardrailCheckResult)`

`→ tripwire = not is_appropriate`

- **문맥·의미 이해, 유연한 기준 적용**
- LLM 호출 1회 추가 — 지연·비용 발생

실습 : 07_Output Guardrail

07. Guardrails (가드레일)

가드레일(Guardrails) 은 에이전트와 함께 실행되며, **입력**과 **출력**에 대해 검사와 유효성 검증을 수행할 수 있게 해줍니다. 조건을 만족하지 않으면 **트립와이어(tripwire)** 를 발동시켜 실행을 조기에 중단할 수 있습니다.

이 노트북에서는 두 가지 가드레일을 순서대로 다룹니다.

1. **Input Guardrail** (`@input_guardrail`) — 에이전트 실행 **전** 사용자 입력을 검증
2. **Output Guardrail** (`@output_guardrail`) — 에이전트 실행 **후** 생성된 출력을 검증

Input vs Output Guardrail 비교

구분	Input Guardrail	Output Guardrail
검증 시점	에이전트 실행 **전**	에이전트 실행 **후**
검증 대상	사용자 입력	에이전트 출력
데코레이터	`@input_guardrail`	`@output_guardrail`
예외	`InputGuardrailTripwireTriggered`	`OutputGuardrailTripwireTriggered`
실행 모드	Parallel(병렬) 또는 Blocking(차단)	항상 Sequential(순차)

Tripwire (트립와이어)란?

가드레일이 문제를 감지했을 때 발생시키는 **즉시 중단 신호**입니다. `tripwire_triggered=True` 로 설정하면 즉시 예외가 발생하고 실행이 중단됩니다.

다양한 시각화 - draw_graph()

draw_graph() 사용법

단일 에이전트 + 도구

```
from agents.extensions.visualization import draw_graph
draw_graph(weather_agent)  # Jupyter 인라인 표시
```

멀티에이전트 (핸드오프)

```
triage = Agent(name="분류", handoffs=[ko_agent, en_agent])
draw_graph(triage)
```

파일 저장 & 브라우저

```
draw_graph(agent, filename="graph")  # png 저장
draw_graph(agent).view()         # 뷰어에서 열기
```

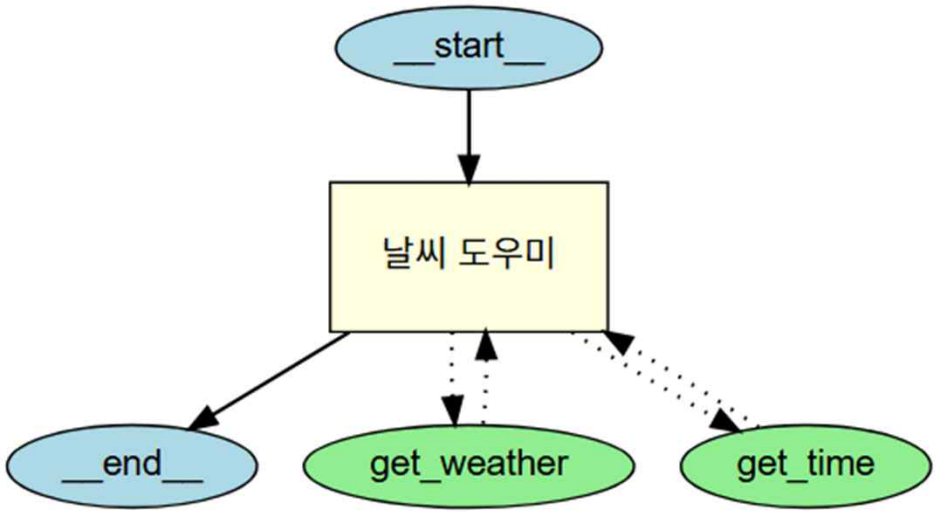
실습 : 08_Agent Visualization

08. Agent Visualization (에이전트 시각화)

OpenAI Agent SDK는 **Graphviz**를 사용하여 에이전트 구조를 시각적으로 표현하는 기능을 제공합니다.

시각화가 보여주는 것:

- **노란색 박스**: 에이전트 (Agent)
- **녹색 타원**: 도구 (Tool)
- **회색 박스**: MCP 서버
- **실선 화살표**: 핸드오프 연결
- **점선 화살표**: 도구 연결
- **대시선 화살표**: MCP 서버 연결



Chatbot RAG Agent

도구 없는 단순 챗봇에서 시작해, 도구를 하나씩 붙여가며 RAG 에이전트를 완성



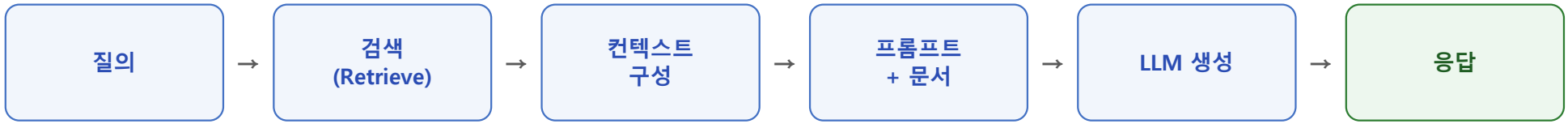
RAG란? : 검색 + 생성의 결합

검색 증강 생성(Retrieval-Augmented Generation) — 외부 지식 베이스를 검색해 LLM 응답의 **정확성과 최신성**을 보완하는 기술

검색기 (Retriever)
사용자 입력과 관련된 정보를 외부 데이터베이스에서 검색

생성기 (Generator)
검색된 문서를 컨텍스트로 활용해 사용자에게 응답을 생성

실행 흐름



- 문서 로드 → 청킹 → 임베딩 → 저장 → 검색 파이프라인을 **OpenAI 관리형 Vector Store**가 자동 처리
- 파일 업로드와 `FileSearchTool` 연결만으로 RAG가 완성됩니다.

Vector Store란? — 의미로 검색하는 데이터베이스

텍스트를 **임베딩 벡터**(의미를 담은 숫자 배열)로 변환해 저장하고, **벡터 간 거리(유사도)**로 관련 문서를 찾는 데이터베이스

"환불 정책이 어떻게 되나요?"
질의 / 문서 텍스트

↓

임베딩 모델
text-embedding-3 등

↓

`[0.21, -0.73, 0.55, …, 1.42]`
고차원 벡터 (수백~수천 차원)

↓

유사도 검색
질의 벡터와 가까운 문서 벡터 탐색



키워드 검색 — 단어 일치 기반

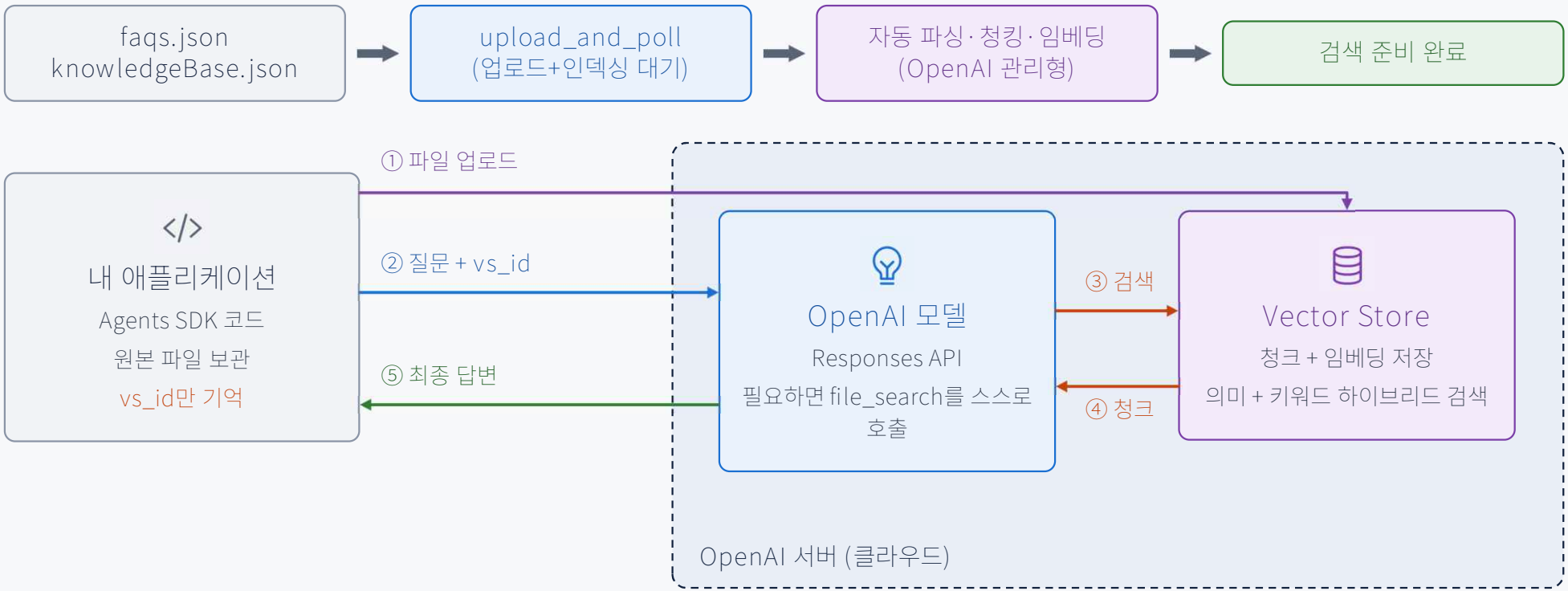
- "환불" ≠ "돈 돌려받기" — 단어가 다르면 못 찾음
- 동의어·표현 변형·오타에 취약

벡터(의미) 검색 — 임베딩 유사도 기반

- 표현이 달라도 의미가 같으면 매칭 (코사인 유사도 등)
- 전용 DB: Chroma·FAISS·Pinecone 등 — 이 과정은 OpenAI 관리형 사용

Vector Store는 OpenAI 서버 안에서 모델 곁에 있습니다

생성 방법 (최초 1회)



실습 : 10_Chatbot_RAG_Agent

10. Chatbot RAG Agent 구현

OpenAI Agents SDK로 채팅 에이전트를 단계별로 확장합니다. 도구 없는 단순 챗봇에서 시작해 날씨/웹검색/RAG 도구를 하나씩 붙여가며 완성합니다.

```
1. 기본 대화          Agent + Runner.run()
      ↓
2. 스트리밍           Runner.run_streamed() + 토큰 단위 출력
      ↓
3. 도구 추가          + get_weather + WebSearchTool
      ↓
4. RAG 추가           + FileSearchTool (Vector Store)
      ↓
5. 웹 UI              → Streamlit app.py
```

단계	내용	핵심 API / 도구
1. 비동기 방식 호출	도구 없이 Agent + Runner로 대화 구현 (메시지 유지)	`await Runner.run()`
2. Streaming 방식 호출	동일 에이전트를 토큰 단위 실시간 출력으로 실행	`Runner.run_streamed()`, `ResponseTextDeltaEvent`
3. 도구를 추가한 Chatbot	날씨 조회 + 웹 검색 도구를 에이전트에 연결	`@function_tool` get_weather, `WebSearchTool()`
4. RAG 도구 추가	OpenAI Vector Store에 FAQ/KB 업로드 후 검색 도구 연결	`FileSearchTool(vector_store_ids=[...])`
5. 웹 UI 연결	동일 에이전트를 Streamlit 웹 앱으로 구현한 예제 안내	`10_chat-app-rag/app.py`

What is Flask ?

- Python으로 작성된 경량(micro) 웹 애플리케이션 프레임워크
- 라우팅·요청·응답 등 웹 서버의 핵심 기능을 데코레이터(@app.route)로 간결하게 정의
- JSON API를 쉽게 만들 수 있어 HTML/JavaScript 프런트엔드와 자유롭게 조합 가능
- 템플릿(Jinja2), 세션, 확장 생태계(DB·인증 등)를 필요한 만큼만 선택해 사용
- 코드 몇 줄로 서버 실행(app.run()) — 학습과 프로토타이핑에 적합
- 소규모 API 서버부터 대규모 서비스까지 폭넓게 활용 (Django보다 가볍고 자유로움)

app.py ↔ index.html — Component 상호 작용 흐름



실행 화면 — localhost:8000

브라우저 — index.html

(바닐라 JavaScript 채팅 UI)

채팅 UI (#chat-log)
user·bot 말풍선 · 자동 스크롤

입력 폼 submit
fetch POST /chat {message}

응답 표시
로딩 말풍선 → data.reply 교체

대화 초기화 버튼
fetch POST /reset → 로그 비움

`GET / → send_file(index.html)`
최초 접속 시 채팅 UI 페이지 반환

1

`POST /chat {"message"}`
사용자 질문 전송 (JSON)

2

② 수신 후: messages 추가
→ Runner.run_sync(agent, history)
→ 도구 호출 → ③ 응답

3

`{"reply": final_output}`
에이전트 실행 결과를 JSON으로 응답

4

`POST /reset → {"ok": true}`
messages.clear() — 대화 기록 초기화

Flask 서버 — app.py

(JSON API + 에이전트 실행)

라우트
GET / · POST /chat · POST /reset

messages + Lock
대화 기록을 서버 메모리에 유지

Agent — Runner.run_sync
instructions + history 입력

Tools
get_weather · WebSearch · FileSearch

OpenAI Vector Store (RAG)
faqs.json · knowledgeBase.json

Flask 실행

- python app.py

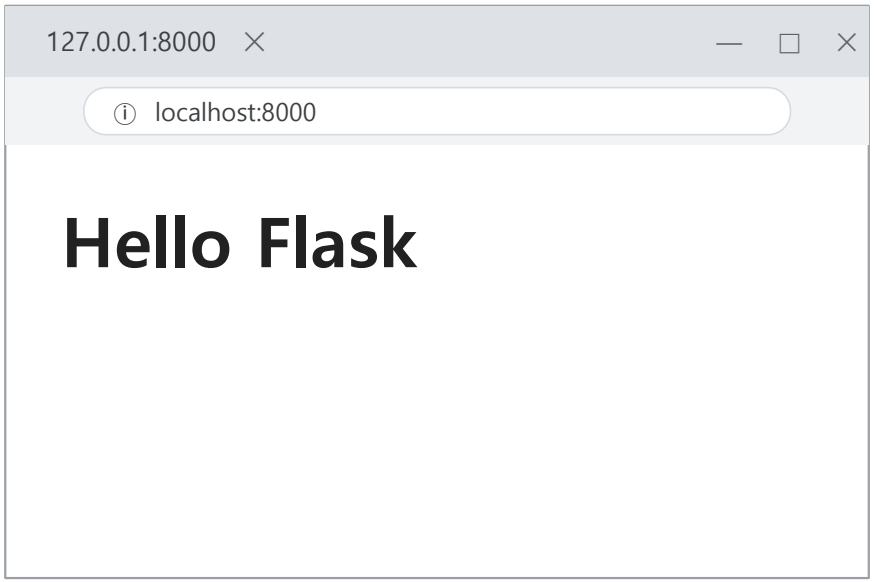
app.py

```python
from flask import Flask

app = Flask(__name__)

@app.route("/")
def home():
    return "<h1>Hello Flask</h1>"

if __name__ == '__main__':
    app.run(port=8000)
```

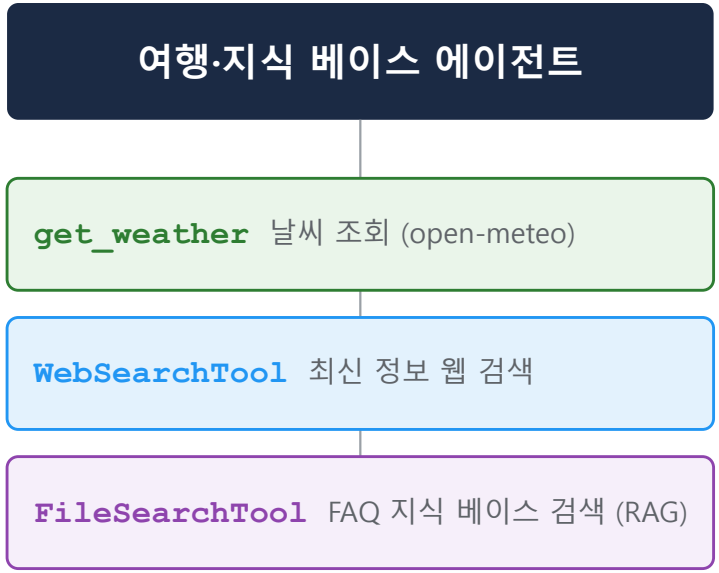



```
(base) C:\...\01_OpenAI_API_Basic\14_rag-chat-flask>python app.py
RAG 챗봇 서버 시작: http://localhost:8000  (종료: Ctrl+C)
 * Serving Flask app 'app'
 * Running on http://127.0.0.1:8000
```

최종 에이전트 구성 + 웹 UI 연결

도구 3개를 통합한 최종 에이전트

```
agent = Agent(
    model=Model,
    name="여행·지식 베이스 에이전트",
    instructions="날씨는 get_weather, 최신 정보는"
        " 웹 검색, FAQ는 file_search로 답변",
    tools=[
        get_weather,                          # ① 날씨
        WebSearchTool(),                      # ② 웹 검색
        FileSearchTool(vector_store_ids=[vs_id]), # ③ RAG
    ],
)
```



웹 UI 연결: 동일한 에이전트 구조를 Flask + HTML 앱으로 구현한 예제
— **11_rag_chat_flask/app.py** (실행: `python app.py` → http://localhost:8000 접속)

실습 : 11_Rag_Chat_Flask_Agent — Flask + HTML

- **OpenAI Agents SDK** 에이전트를 웹으로 제공
- Flask가 JSON API를 담당하고, HTML(바닐라 JavaScript)이 채팅 UI를 담당

단계	내용	핵심 API / 도구
1. Vector Store 설정	서버 시작 시 1회 — 같은 이름(notebook-faq-kb) 있으면 재사용, 없으면 생성 후 FAQ/KB 업로드	`vector_stores.create()` `file_batches.upload_and_poll()`
2. 날씨 도구	위도·경도로 현재 기온 조회 (open-meteo API)	`@function_tool get_weather`
3. 에이전트 생성	서버 시작 시 1회 생성해 재사용 — 도구 3개 연결 (모델: gpt-5.4-mini)	`Agent, WebSearchTool(),` `FileSearchTool(vector_store_ids=[vs_id])`
4. 채팅 API	메시지를 대화 기록에 추가 후 전체 기록으로 에이전트 실행, 답변을 JSON으로 반환	`POST /chat, Runner.run_sync()`
5. 대화 상태 관리	서버 메모리 리스트 + Lock으로 보호 (st.session_state 대응) — /reset으로 초기화	`threading.Lock, POST /reset`

멀티에이전트 오케스트레이션 — 코드 기반 제어

LLM 기반(Handoffs) vs 코드 기반 오케스트레이션

구분	LLM 기반 (Handoffs)	코드 기반
제어 주체	LLM이 자율 판단	개발자가 코드로 제어
예측 가능성	낮음 (LLM 판단에 의존)	높음 (결정적 흐름)
비용 / 속도	변동적	예측 가능
적합한 경우	복잡한 대화, 분류가 어려운 경우	파이프라인, 배치 처리, 정해진 워크플로우

4가지 패턴의 Multi-Agent Orchestration

1. 순차 체이닝
A → B → C 파이프라인

2. 병렬 실행
asyncio.gather()

3. 분류 → 라우팅
Structured Output

4. 조합 패턴
병렬 + 순차 결합

멀티 에이전트 오케스트레이션 — 4가지 패턴 요약

LLM이 아니라 코드가 흐름을 제어하는 워크플로우 패턴

의존성이 있으면 순차, 없으면 병렬, 분기가 필요하면 분류 + 라우팅 — 실전은 이들을 결합

① 순차 체이닝 Sequential Chaining

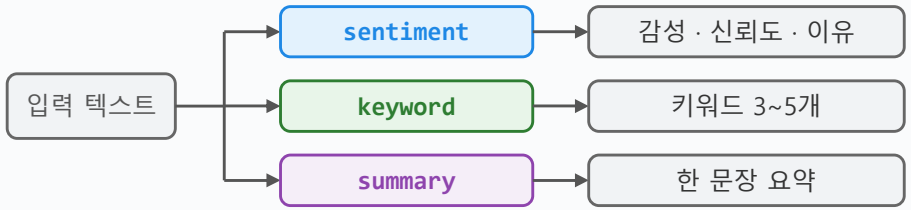


이전 단계의 final_output → 다음 에이전트의 입력

핵심 `Runner.run(next, prev.final_output)`

사용 시점 각 단계가 앞 단계 결과에 의존할 때 (초안 → 편집 → 번역)

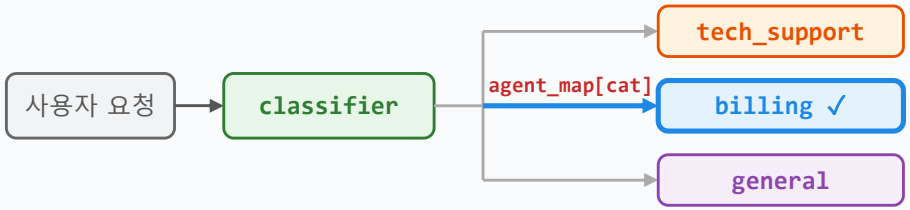
② 병렬 실행 Parallelization



핵심 `await asyncio.gather(*[Runner.run(a, text) ...])`

사용 시점 서로 독립적인 작업 → 전체 대기 시간 ≈ 가장 느린 1개

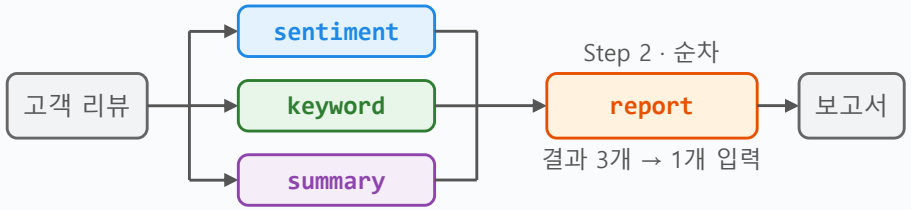
③ 분류 + 라우팅 Classification + Routing



핵심 `output_type`으로 구조화 분류 → 코드가 `agent_map`에서 선택

사용 시점 Handoffs와 달리 결정적(deterministic) 분기가 필요할 때

④ 병렬 분석 + 순차 종합 Combined



핵심 Step 1 `asyncio.gather` → Step 2 `Runner.run(report, ...)`

사용 시점 실전 워크플로우 — 여러 패턴을 조합해 구성

실습 : 12_multi_agent_orchestration

12. Multi-Agent Orchestration (코드 기반 멀티에이전트 오케스트레이션)

04_Handoffs에서는 LLM이 자율적으로 핸드오프를 결정하는 **LLM 기반 오케스트레이션**을 배웠습니다. 이번에는 **코드로 직접** 에이전트 흐름을 제어하는 패턴을 학습합니다.

LLM 기반 vs 코드 기반 오케스트레이션

구분	LLM 기반 (Handoffs)	코드 기반
제어 주체	LLM이 자율 판단	개발자가 코드로 제어
예측 가능성	낮음 (LLM 판단에 의존)	높음 (결정적 흐름)
유연성	높음	보통
비용/속도	변동적	예측 가능
적합한 경우	복잡한 대화, 분류가 어려운 경우	파이프라인, 배치 처리, 정해진 워크플로우

주요 패턴

1. **순차 체이닝 (Sequential Chaining)**: A → B → C
2. **병렬 실행 (Parallelization)**: A, B, C 동시 실행
3. **분류 → 라우팅 (Classification + Routing)**: 분류 후 전문 에이전트에 전달

MCP
(Model Context Protocol)

MCP 구현 구성 요소

MCP 클라이언트
앱 · 에이전트

연결 · 요청 →
← 결과

MCP 서버
의류 재고 + Chinook

서버가 아래 세 가지를 제공하고,
클라이언트가 연결해서 꺼내 쓴다

	Resources (리소스)	Tools (툴)	Prompts (프롬프트)
역할	AI 모델에게 줄 정보 · 컨텍스트	모델이 외부 시스템에서 하는 작업	재사용하는 메시지 템플릿
누가 고르나	**앱이 골라 읽는다**	**모델이 골라 호출한다**	**사용자가 슬래시 명령 · 메뉴로 고른다**
REST로 치면	GET	POST · PUT · DELETE	미리 써 둔 요청 양식
부작용	없음 (읽기 전용)	있을 수 있음	없음 (메시지만 만든다)
예	고객 프로필 · 제품 목록 · DB 값 읽기	데이터 생성 · 수정 · 이메일 발송 · 결제	• 자주 쓰이는 질문/명령을 표준화 • 일관성 있는 요청을 보장 예) "사용자 ID로 주문 목록 조회" 표준 질문

한 줄 요약: 셋의 차이는 **누가 고르느냐** — **앱**이 읽는 Resources · **모델**이 부르는 Tools · **사용자**가 고르는 Prompts

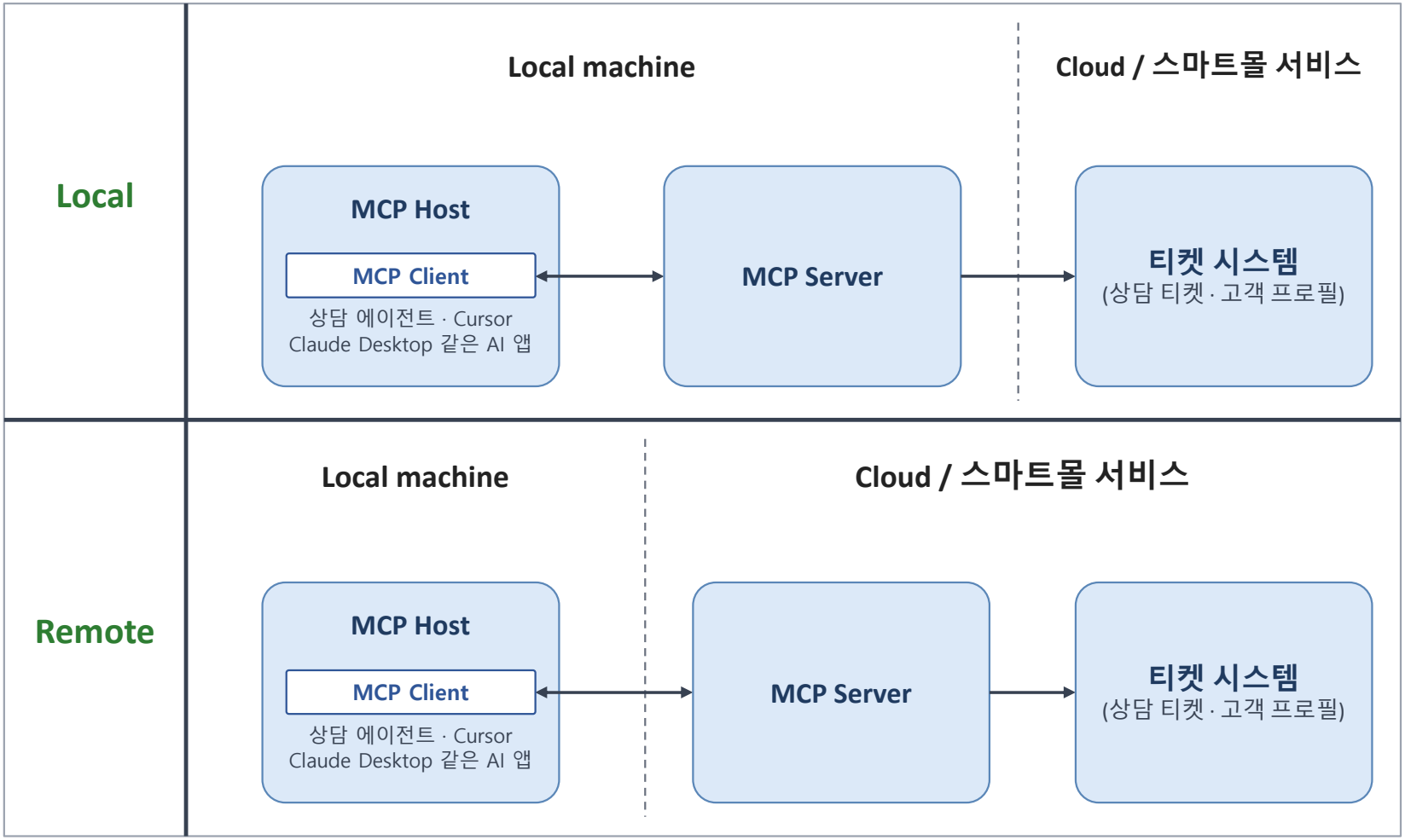
MCP 표준 전송 유형

- **Stdio (Standard In/Output)**
  - 클라이언트가 MCP 서버를 subprocess로 실행하고, 표준 입출력(stdin/stdout)으로 JSON-RPC 메시지를 교환
  - 서버를 로컬 머신에서 실행할 때 사용 — 네트워크 설정 없이 같은 PC 안에서 연결

- **Streamable HTTP**
  - 원격(Remote) MCP 서버용 표준 전송 — 단일 HTTP 엔드포인트(예: https://example.com/mcp)에 메시지마다 POST · 응답은 JSON 또는 SSE 스트림
  - 프로토콜 수준 세션이 없는 상태 없는(stateless) 방식 — 어느 서버 인스턴스가 받아도 되어 수평 확장에 유리 (Mcp-Session-Id 세션 · GET 스트림은 이전 규격)

기준: MCP 규격 2026-07-28 개정판 (modelcontextprotocol.io)

Local/Remote MCP



Stdio (Standard In/Output)

같은 PC에서 하위 프로세스로 실행
보통 클라이언트 1개

Streamable HTTP

네트워크로 연결
보통 클라이언트 여러 개
로컬에서도 가능 (실습 10은 127.0.0.1:8000)

MCP 파이썬 SDK

- SDK (Software Development Kit)
  - 개발자가 특정 플랫폼용 앱을 개발할 수 있도록 하는 도구 모음
- 모든 MCP 서버에 연결할 수 있는 MCP 클라이언트를 구축할 수 있음
- 리소스, 프롬프트 및 도구를 노출하는 MCP 서버를 만들 수 있음
- stdio, Streamable HTTP와 같은 표준 전송을 사용
- 모든 MCP 프로토콜 메시지 및 수명 주기 이벤트를 처리

설치

```
pip install mcp
```

공식 MCP Python SDK — 클라이언트 · 서버 · 표준 전송

```
pip install fastmcp
```

이 과정의 서버 코드가 쓰는 프레임워크 (from fastmcp import FastMCP)

이 과정에서는 `pip install -r requirements.txt` 한 번으로 둘 다 설치됨 (openai-agents도 mcp를 의존성으로 함께 설치)

MCP Server

```python
from mcp.server.fastmcp import FastMCP

# Create an MCP server
mcp = FastMCP("Demo")


# Add an addition tool
@mcp.tool()
def add(a: int, b: int) -> int:
    """Add two numbers"""
    return a + b


# Add a dynamic greeting resource
@mcp.resource("greeting://{name}")
def get_greeting(name: str) -> str:
    """Get a personalized greeting"""
    return f"Hello, {name}!"
```

MCP server 지정

MCP 도구 지정

MCP가 가지고 있는 data, configuration 등 정의

MCP Client

```python
if transport == "stdio":
    # mcp.json 읽기 → 서버를 subprocess로 기동
    with open(BASE_DIR / "mcp.json", "r", encoding="utf-8") as f:
        server_config = json.load(f)["mcpServers"]["mcp_server"]

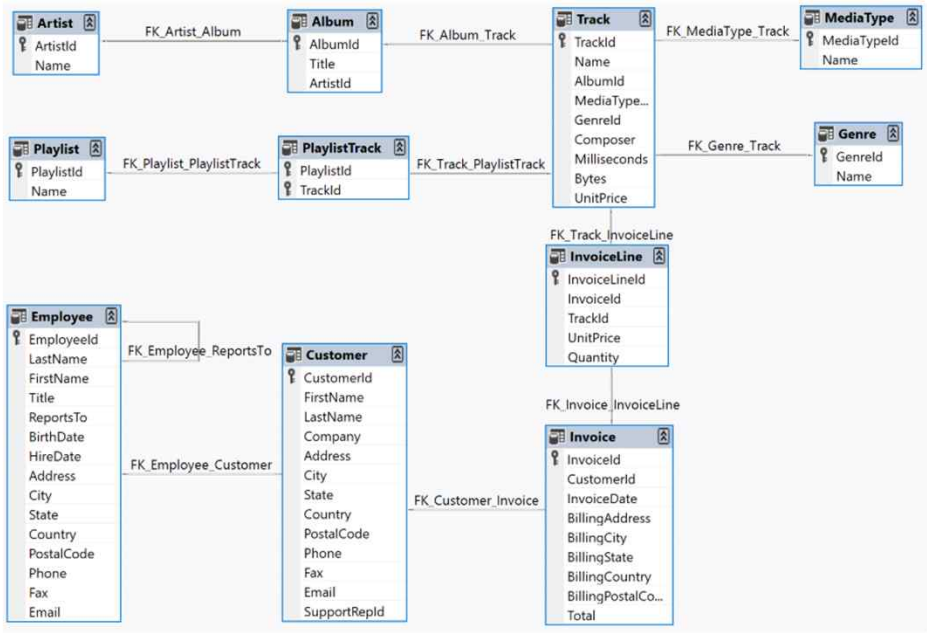
    command = server_config["command"]
    # 지금 클라이언트를 실행 중인 인터프리터(sys.executable)로 서버 기동
    if command == "python":
        command = sys.executable
    return MCPServerStdio(
        params={
            "command": command,
            "args": server_config.get("args", []),
            "cwd": str(BASE_DIR),                    # 서버 스크립트 위치 기준 실행
        },
        **common,
    )

if transport == "http":
    # 이미 실행 중인 HTTP 서버에 URL로 접속
    return MCPServerStreamableHttp(params={"url": HTTP_SERVER_URL}, **common)
```

MCPClient 설정

Chinook Database

- 관계형 DB를 학습하고 실습할 수 있도록 제공되는 샘플 DB
- 음악 스토어(Music Store)를 모델링한 데이터베이스



- 현재 등록된 모든 의류 목록을 보여줘.
- 신제품인 glove 를 5 달러로 새로 등록해줘.

- DB Resource 테스트 질문
  - **1. Database Info Resource:**
    - 데이터베이스 정보를 보여줘"
    - "Chinook 데이터베이스에 대해 알려줘"
    - "데이터베이스 기본 정보를 확인해줘"
  - **2. Table Info Resource:**
    - "Employee 테이블 정보를 보여줘"
    - "Customer 테이블의 구조를 알려줘"
    - "Album 테이블의 스키마를 확인해줘"

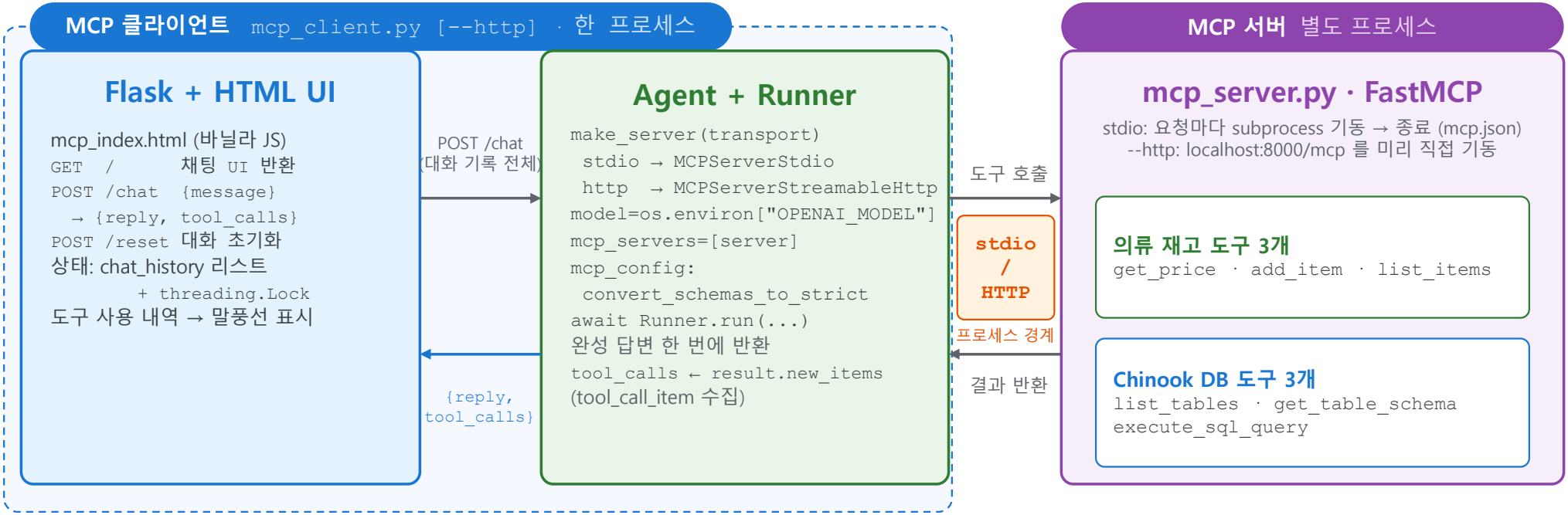
- DB Tools 테스트 질문
  - **1. SQL Query Execution Tool:**
    - "직원이 몇 명인가요?"
    - "고객 수를 알려줘"
    - "앨범이 총 몇 개인가요?"
    - "가장 많이 팔린 곡은 무엇인가요?"
    - "매니저는 몇 명인가요?"
  - **2. Table Schema Tool:**
    - "Employee 테이블의 구조를 보여줘"
    - "Customer 테이블의 컬럼 정보를 알려줘"
    - "Track 테이블의 스키마를 확인해줘"
  - **3. List Tables Tool:**
    - "데이터베이스에 어떤 테이블들이 있나요?"
    - "사용 가능한 테이블 목록을 보여줘"
    - "모든 테이블 이름을 알려줘"

- **연속 대화 테스트:**
  - 1단계: "직원이 몇 명인가요?"
  - 2단계: "그 중에서 매니저는 몇 명인가요?"

- **컨텍스트 참조 테스트:**
  - 1단계: "고객이 몇 명인가요?"
  - 2단계: "그 고객들 중에서 미국 고객은 몇 명인가요?"
  - 3단계: "앞서 말한 고객들의 주문 건수는?"

실습 : MCP 클라이언트 (stdio / Streamable HTTP)

Hands-on



실행 (stdio): `python mcp_client.py` → http://localhost:8001 접속
실행 (--http): ① `python mcp_server.py --http` **(port 8000)** ② `python mcp_client.py --http` → http://localhost:8001 접속

서버 실행

① 터미널 1 — MCP 서버 기동 (HTTP 모드)

```
cd 02_OpenAI_MCP_Agent
python mcp_server.py --http
```

→ "Chinook 데이터베이스 연결 성공" 출력 후
http://127.0.0.1:8000/mcp 에서 요청 대기 (창은 닫지 않음)

② 터미널 2 — MCP 클라이언트(Flask) 기동

```
python mcp_client.py --http
```

→ .env의 OPENAI_API_KEY, OPENAI_MODEL 사용
http://localhost:8001 에서 웹 UI 대기

③ 브라우저 — http://localhost:8001 접속

예) "jeans 가격 알려줘", "Artist 테이블에서 5개만 보여줘"
→ 답변과 함께 사용한 도구(tool_calls)가 표시됨

④ 종료 — 각 터미널에서 Ctrl+C (클라이언트 → 서버 순)

참고 stdio 모드는 ①이 필요 없음: `python mcp_client.py` 만 실행하면 요청마다 서버를 subprocess로 자동 기동




```
(langchain) C:\Users\trimu\OneDrive\Agentic_AI_GPT_MCP_LangChain\02_OpenAI_MCP_Agent>python mcp_client.py

MCP 챗봇 서버 시작 (stdio): http://localhost:8001  (종료: Ctrl+C)
 * Serving Flask app 'mcp_client'
 * Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server
instead.
 * Running on http://127.0.0.1:8001
Press CTRL+C to quit
127.0.0.1 - - [01/Nov/2026 15:02:25] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [01/Nov/2026 15:02:49] "POST /chat HTTP/1.1" 200 -
```

LangChain Agent

LangChain이란?

- 대규모 언어 모델(LLM)을 활용한 애플리케이션 개발 프레임워크
- LLM 애플리케이션의 전체 라이프사이클을 간소화
- Python 및 TypeScript 지원
- **LangChain** - 원하는 모델 제공자를 사용해 에이전트를 빠르고 손쉽게 구축
- **LangGraph** - 에이전트의 모든 단계를 세밀하게 제어하는 저수준 오케스트레이션, 메모리, Human-in-the-loop
- **DeepAgents** - 계획(Planning), 서브에이전트, 파일시스템, 장기 메모리 내장 에이전트 하네스

구분	LangChain	LangGraph	DeepAgents	Managed Deep Agents (LangSmith)
성격	오픈소스 라이브러리	오픈소스 라이브러리	오픈소스 라이브러리 (에이전트 하네스)	관리형 서비스 (클라우드 플랫폼)
주요 목적	LLM 애플리케이션 개발을 쉽고 빠르게	에이전트 워크플로우를 세밀하게 제어	복잡한 작업을 수행하는 고급 에이전트 구축	DeepAgents를 손쉽게 배포하고 운영
핵심 기능	모델 연동, 프롬프트, 체인, 검색(RAG), 도구 연동 등	그래프 기반 오케스트레이션, 메모리, 상태 관리, Human-in-the-loop	계획(Planning), 서브에이전트, 파일 시스템, 장기 메모리, 스킬	에이전트 실행 인프라 관리 (서버, 메모리, 스케줄링, 채널 등)
실행 방식	개발자가 직접 구축/실행	개발자가 직접 구축/실행	개발자가 직접 구축/실행	LangSmith가 관리형 런타임에서 실행
제공 범위	빌딩 블록(컴포넌트)	워크플로우 엔진	완성도 높은 에이전트 런타임	에이전트 정의 + 실행 인프라 + 운영 기능
대표 사용 사례	RAG 챗봇, 문서 요약, 도구 연동 애플리케이션	복잡한 멀티스텝 에이전트, 승인 워크플로우, 상태 관리가 필요한 서비스	장시간 작업, 다단계 연구/분석, 서브에이전트 협업, 코드 실행	프로덕션 배포, 팀 협업, 모니터링, 스케줄링, Slack 등 채널 연동

LangChain · LangGraph · Deep Agents — 3계층 구조

Learn

Deep Agents

Harness

계획 · 파일시스템 · 서브에이전트 · 스킬이 내장된 에이전트

LangChain

Framework

create_agent · 미들웨어 · 모델/도구 표준 인터페이스

LangGraph

Runtime

State 그래프 · 지속 실행 · 스트리밍 · 체크포인트 · interrupt

위 계층은 아래 계층 위에서 동작한다 — 결과물은 모두 LangGraph 그래프

언제 무엇을 쓰나

- Deep Agents: 개방형 · 장시간 멀티스텝 작업 (리서치, 코딩)
- LangChain: 표준 도구 호출 에이전트를 빠르게 구축
- LangGraph: 에이전트에 검증 · 승인 · 기억 · 팀 구성을 직접 설계해 넣을 때

출처: LangChain 공식 문서 — Frameworks, runtimes, and harnesses

이 과정에서 Deep Agents를 다루지 않는 이유 — 하네스를 쓰기 전에, 하네스 안에 든 것을 직접 만들 줄 알아야 한다

1 이미 직접 만드는 기능의 묶음

계획·서브에이전트·스킬·승인(HITL)은 이 과정의 310 · 340 · 104 실습에서 create_agent로 하나씩 구현한다.

2 안이 감춰진 완성품

프롬프트 · 도구 · 실행 흐름이 미리 정해져 있어 "왜 이렇게 동작하는지" 보기 어렵다.

3 아직 빠르게 바뀌는 API

0.x 단계로 버전마다 사용법이 달라진다

과정을 마친 뒤 Deep Agents를 쓰면, 그 안의 계획 · 서브에이전트 · 스킬 · interrupt_on이 모두 이미 배운 패턴임을 알아볼 수 있다

LangChain 주요 개념 (Core Components)

1. **Agents (에이전트)** - 사용자 입력에 따라 도구를 선택하고 실행하는 자율적 시스템
2. **Models (모델)** - 채팅에 특화된 LLM (Chat Models), 다양한 제공자 통합 지원
3. **Messages (메시지)** - 채팅 모델의 입력과 출력 단위 (Human, AI, System, Tool)
4. **Tools (도구)** - 모델이 호출할 함수 정의 및 Tool Calling 기능
5. **Short-term memory (단기 메모리)** - 대화 기록을 메시지 시퀀스로 관리
6. **Streaming (스트리밍)** - 모델 응답을 토큰 단위로 실시간 전달
7. **Structured output (구조화된 출력)** - JSON 등 지정 스키마에 맞춰 응답

LangChain Messages

메시지 구성 요소

Role (역할) → 메시지의 유형 식별,
예: system, user, assistant 등
Content (내용) → 메시지의 실제 내용 (텍스트, 이미지, 오디오 등)
Metadata (메타데이터) → 선택적(optional) 필드. 응답 정보(response info), 메시지 ID, 토큰 사용량(token usage) 등

메시지 유형 (Message types)

System message (시스템 메시지)
Human message (사용자 메시지)
AI message (AI 메시지)
Tool message (도구 메시지)

```python
from langchain_core.messages import SystemMessage, HumanMessage

msgs = [
  SystemMessage(content="너는 친절한 튜터야"),
  HumanMessage(content="RAG를 3줄로 설명해줘"),
]
resp = llm.invoke(msgs)
print(resp.content)

```

Langgraph 란?

- LLM을 활용해 **상태 기반의** 다중 **에이전트 애플리케이션**을 구축
- 그래프 구조를 통해 복잡한 워크플로우를 효율적으로 설계 및 관리
- 상태를 지속적으로 유지해 반복 작업과 복잡한 상태 관리를 지원
- 인간의 개입이 필요한 워크플로우와 메모리 기능을 유연하게 구현

주요 특징

  - 상태 지속성
  - 워크플로우 최적화
  - 복잡한 작업 처리
- **활용 사례** - 고객 서비스 챗봇, 복잡한 데이터 분석, 교육용 튜터링 시스템 등

3가지 핵심 구성 요소 — State, Node, Edge

State — 공유 데이터 구조

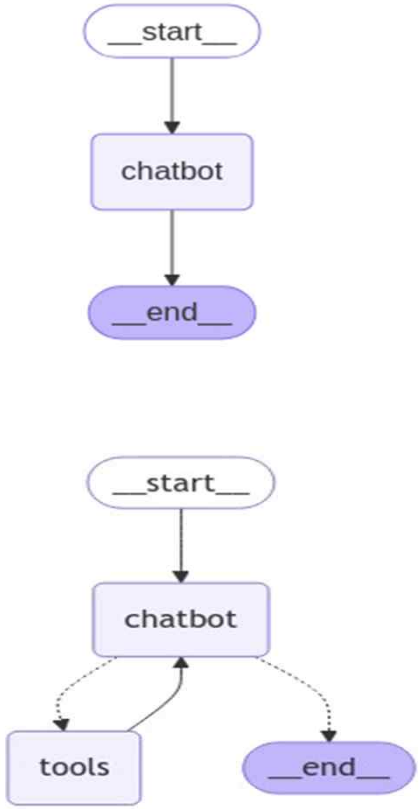
- 애플리케이션의 현재 스냅샷을 나타내는 공유 데이터
- 노드 간 전달되며, 각 노드가 읽고 업데이트
- StateSchema 로 정의

Node — 작업의 단위

- 에이전트 로직을 인코딩하는 함수 (GraphNode)
- 현재 상태를 입력받아 계산 수행 후 업데이트된 상태 반환
- LLM 호출 또는 일반 코드 모두 가능

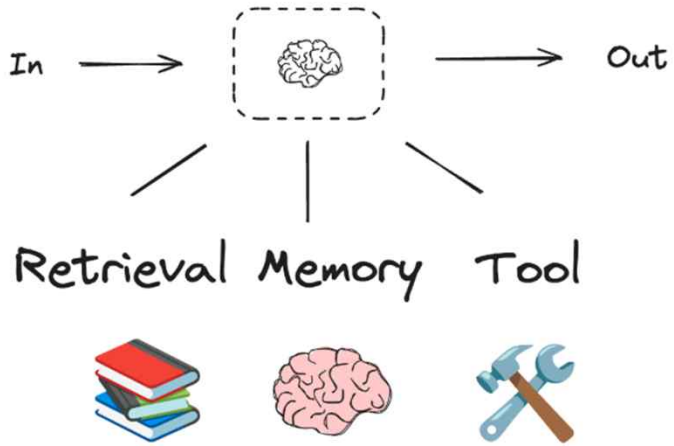
Edge — 흐름 제어

- 현재 상태에 따라 다음 실행할 노드를 결정하는 함수
- **고정 엣지**(직접 연결) 또는 **조건부 엣지**(동적 분기)



Agent의 핵심 기술 스택

- LLM (예: GPT, Claude): 언어 이해 및 생성
- LangChain / LangGraph: 에이전트 워크플로우 및 상태 관리
- Vector Database (예: Chroma, Pinecone): 정보 저장 및 검색
- Memory: 상태 유지 및 맥락 관리
- API 통합 (예: Tavily, SerpAPI): 외부 도구와 연결



Agent 생성 & 상태 관리

create_agent()
- model + tools → 즉시 실행 가능한 에이전트
- system prompt로 에이전트 역할 지정
- invoke() / stream() 두 가지 실행 방식

에이전트 상태(AgentState)
- messages: 대화 기록 자동 누적
- thread_id로 대화 세션 분리
- checkpointer 연결 시 상태 영속화

Agent 실행 흐름
- 1. 사용자 입력 수신
- 2. LLM 추론 (Reason)
- 3. 도구 호출 (Action)
- 4. 결과 관찰 (Observe)
- 5. 완료 판단 → 응답 반환

상태 관리 포인트
- config={"thread_id": "abc"}
- get_state()로 현재 상태 조회

도구 속성 커스터마이징 & Pydantic 스키마

도구 속성 재정의

- name, description 커스터마이징으로 LLM 이해도 향상
- args_schema로 입력 파라미터 타입/제약 명시
- return_direct=True로 에이전트 루프 즉시 종료 가능

Pydantic 스키마 활용

- BaseModel 상속으로 도구 입력 구조 정의
- Field(description=...) 으로 각 필드 설명 추가
- 타입 검증 자동화 → LLM 호출 오류 감소

구조화된 출력 (Structured Output)

- LangChain은 with_structured_output() 메서드를 제공하여 스키마를 모델에 바인딩하고, 모델이 구조화된 출력을 생성하도록 지원

핵심 패턴

- @tool 데코레이터: 함수→도구 변환
- StructuredTool: 클래스 기반 정의
- bind_tools(): 모델에 도구 바인딩

코드 예시

```
class SearchInput(BaseModel):
  query: str = Field(
    description="search term"
  )
```

실습 : 101_Tools & Agents

Tools & Agents 기초

이 노트북에서는 LangChain의 도구(Tools)와 에이전트(Agents) 개념을 학습합니다. 도구는 모델이 외부 세계와 상호작용할 수 있게 해주며, 에이전트는 도구를 사용하여 복잡한 작업을 수행합니다.

참고:

- LangChain 공식 문서 - Tools
- LangChain 공식 문서 - Agents

Memory: “대화”를 상태로 다루기

단기 메모리(Short-term Memory)

- AI가 이전 상호작용의 정보를 기억하는 시스템
- 에이전트가 여러 번의 사용자 상호작용을 포함한 복잡한 작업을 수행가능
- 에이전트는 AgentState 를 사용하여 단기 메모리(short-term memory) 를 관리합니다. 특히, 대화 이력(conversation history)은 messages 키를 통해 저장

장기 메모리(Long-term))

- 사용자 선호/프로필/지식 조각을 JSON 형태로 별도 Store에 보관
- retrieval로 필요할 때만 불러오기
- 개인정보/보안 정책이 핵심

InMemorySaver + SqliteSaver

InMemorySaver (단기 메모리)

- Python 딕셔너리 기반 인메모리 저장
- 프로세스 종료 시 데이터 소실
- 개발/테스트 환경에 적합

SqliteSaver (영속 메모리)

- SQLite DB 파일 기반 영구 저장
- 서버 재시작 후에도 대화 복원
- 프로덕션 환경에 적합

비교표

InMemorySaver
- 저장: RAM | 영속: X | 속도: 빠름
- 용도: 개발/테스트

SqliteSaver
- 저장: 파일 | 영속: O | 속도: 중간
- 용도: 프로덕션

PostgresSaver
- 저장: DB서버 | 영속: O | 속도: 중간
- 용도: 대규모 프로덕션

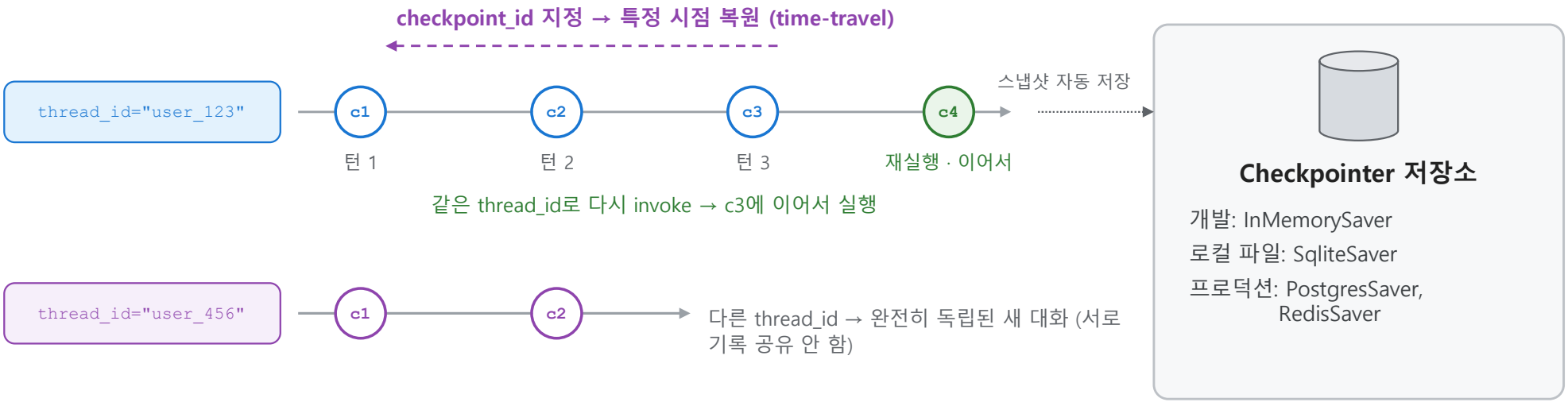
Persistence 상세 — thread_id / checkpoint_id 동작 원리

Thread = 대화 격리 단위

- thread_id: 각 실행 흐름(대화)을 식별하는 고유 ID
- 같은 thread_id → 이전 상태 이어서 실행
- 다른 thread_id → 완전히 독립된 새 대화
- **멀티유저 = 사용자별 thread_id 할당**

Checkpoint = 상태 스냅샷

- 매 super-step마다 자동 저장
- checkpoint_id로 특정 시점 상태 복원 가능
- 개발: InMemorySaver · 로컬: SqliteSaver
- 프로덕션: PostgresSaver / RedisSaver



실습 : 102_Memory Concepts

Memory Concepts (메모리 개념)

메모리(Memory) 는 AI 에이전트가 대화 이력과 상태를 저장하고 관리하는 시스템입니다.

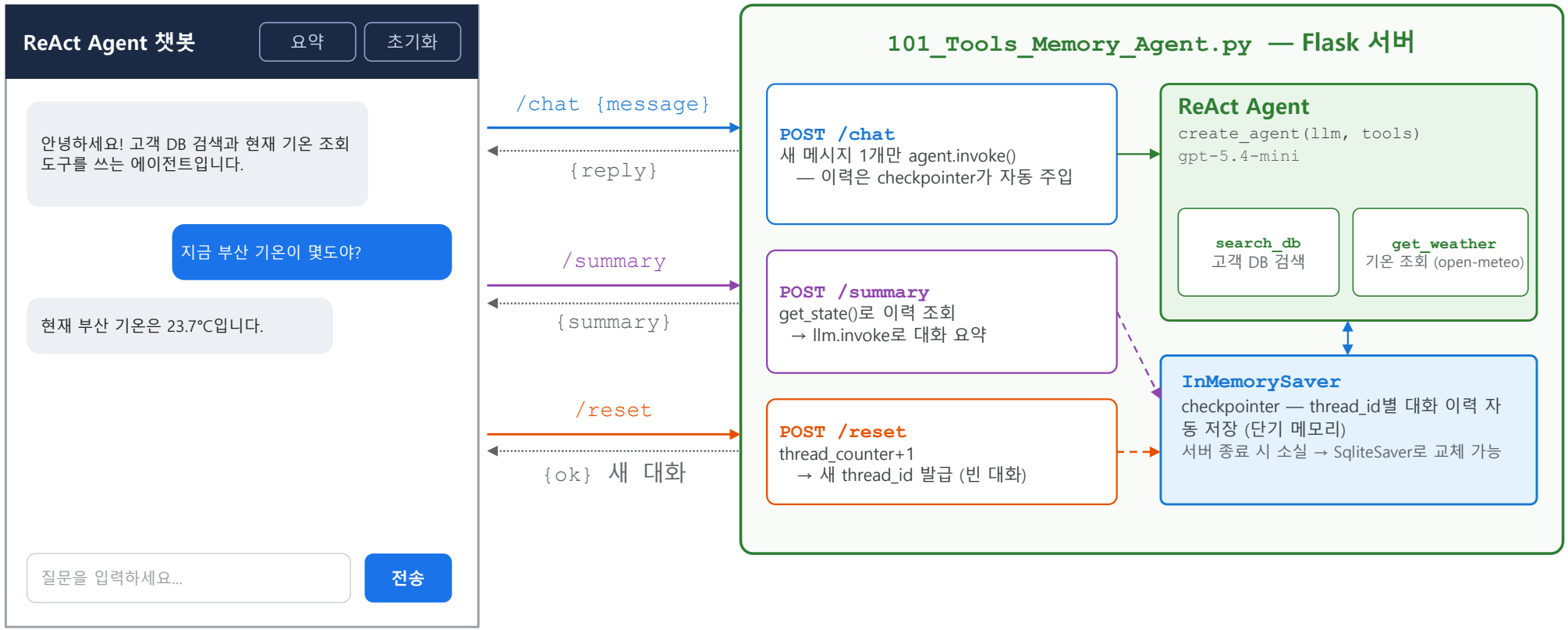
메모리는 두 가지 주요 유형으로 분류됩니다:

- **단기 메모리(Short-term Memory)**: 하나의 대화 세션(thread) 내에서 이전 상호작용을 기억
- **장기 메모리(Long-term Memory)**: 여러 대화 세션에 걸쳐 정보를 유지

이 노트북에서는 메모리의 기본 개념과 사용법을 다룹니다.

참고: LangChain 공식 문서 - Memory

실습 : Flask_WebUI / 101_Tools_Memory_Agent.py



실행: `python 101_Tools_Memory_Agent.py` → http://localhost:8000

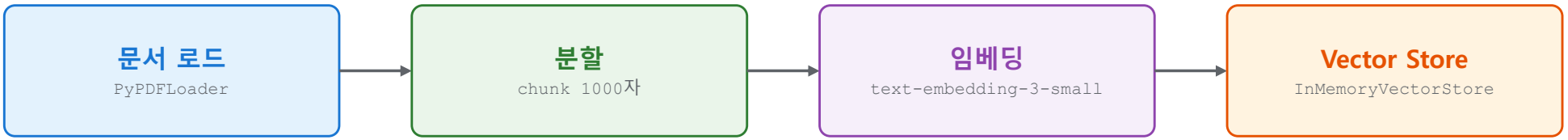
핵심: 매 요청에 새 메시지 1개만 전달 — 이력은 checkpointer가 자동 관리

Vector Store 생성 + Semantic 검색

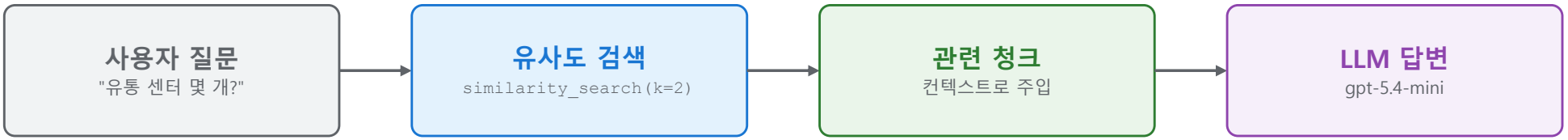
의미(임베딩 유사도) 기반 검색 — 키워드 매칭이 아닌 벡터 거리로 관련 문서를 찾는다

예) 나이키 10-K 보고서 PDF를 지식 베이스로 만들어 에이전트가 검색하게 한다.

① 인덱싱 (최초 1회)



② 검색 + 생성 (질문마다)



검색 품질 3요소 ① 분할 — 의미 단위 청크 ② Retriever 전략 — similarity/MMR·k값 ③ 프롬프트 — 검색 결과 사용법 지시

문서 로드와 분할 (Loaders & Splitters)

페이지 단위는 검색에 너무 크다 — 의미 단위 청크로 분할해야 정확히 검색된다

문서 로드 — 소스를 Document 객체로

PDF · 웹페이지 · CSV · DB 등 소스별 로더 제공

```
loader = PyPDFLoader("nke-10k-2023_korean.pdf")
docs = loader.load()    # 페이지당 Document 1개
docs[0].metadata        # {source, page, ...}
```

텍스트 분할 — RecursiveCharacterTextSplitter

```
splitter = RecursiveCharacterTextSplitter(
    chunk_size=1000,    # 청크 최대 1000자
    chunk_overlap=200,  # 200자 중첩
    add_start_index=True)
```

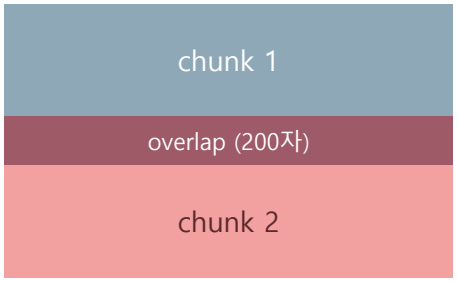
분할 전략 3가지

길이 기준 고정 크기 — 간단·일관

텍스트 구조 단락·문장 단위 — 의미 일관성 (실습 방식)

문서 구조 HTML·Markdown 구조 보존

Chunk Overlap — 경계에서 문맥이 끊기지 않도록 중첩



경계에 걸린 문장도 두 청크 중 하나에는 온전히 포함 → 검색 누락 방지

임베딩과 Vector Store

텍스트 → 벡터 변환 후 저장 — 의미가 비슷하면 벡터 공간에서 가깝다

어떤 Vector Store를 쓸까?

	InMemoryVectorStore	Chroma
용도	실습·소규모 — 이 노트북	영구 저장·대규모
저장	메모리 — 종료 시 소실	persist_directory로 디스크 저장
설치	불필요 — 코어에 내장	chromadb + langchain-chroma

교체는 한 줄: `Chroma(embedding_function=embeddings, persist_directory="./db")`

호스팅형(Pinecone) · 독립형(FAISS, Chroma) · 인메모리형 — 규모와 영속성 요구로 선택

검색을 에이전트에 연결 — Agentic vs Chain RAG

벡터 스토어 검색을 도구(tool)로 감싸면 에이전트가 필요할 때 스스로 호출한다

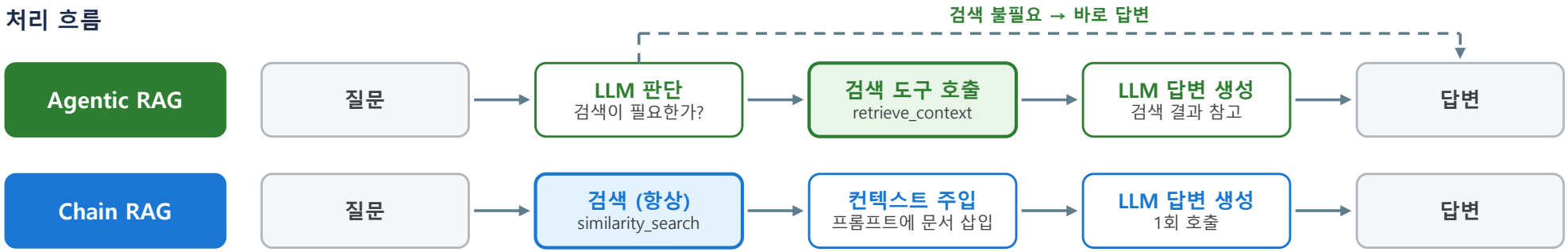
```python
@tool(response_format="content_and_artifact")
def retrieve_context(query: str):
    """질문 관련 정보를 검색합니다."""
    docs = vs.similarity_search(query, k=2)
    return serialized, docs

agent = create_agent(model, [retrieve_context],
                     system_prompt="먼저 검색 후 답변")
```

`content_and_artifact` — LLM에게 줄 문자열과 원본 Document 객체를 함께 반환

구분	Agentic RAG (이 실습)	Chain 기반 RAG
검색 시점	필요할 때만 도구 호출	항상 검색 먼저 실행
모델 호출	여러 번 (판단 → 답변)	1회 (검색 결과 주입)
유연성	높음 — 바로 답 가능	낮음 — 모든 질문에 검색
속도·예측성	지연 시간 증가, 흐름 가변	빠르고 예측 가능

처리 흐름



선택 기준: 대화형·질문 다양 → Agentic / FAQ처럼 항상 검색이 필요 → Chain

Streaming: UX를 바꾸는 기능

스트리밍이 중요한 이유

- 응답 지연이 길수록 체감 품질 급락 — 완성 전에 점진적으로 표시
- 토큰/단계 단위로 "진행 중"을 보여주면 신뢰 상승

stream_mode — 무엇을 스트리밍할지 선택

values
매 스텝 후 전체 상태 스냅샷 — 실습 기본값

updates
각 스텝의 변경분만 — 노드별 진행 상황

messages
LLM 토큰 단위 — 실시간 타이핑 효과

실시간 챗봇

조합 가능: `stream_mode=["updates", "messages"]`
토큰 스트리밍 + 상태 추적 동시 수신

```
for event in agent.stream(
    {"messages": [{"role": "user",
                   "content": "질문"}]},
    config,              # thread_id
    stream_mode="values",
):
    event["messages"][-1].pretty_print()
```

실습 : 103_RAG_Agent 작성

RAG Agent (검색 증강 생성 에이전트)

RAG (Retrieval-Augmented Generation) 는 외부 지식 소스에서 정보를 검색하여 LLM의 응답을 향상시키는 기술입니다.

RAG 에이전트는 검색 도구를 사용하여 관련 문서를 찾고, 이를 컨텍스트로 활용하여 답변을 생성합니다.

참고: LangChain 공식 문서 - RAG Agent

RAG 워크플로우

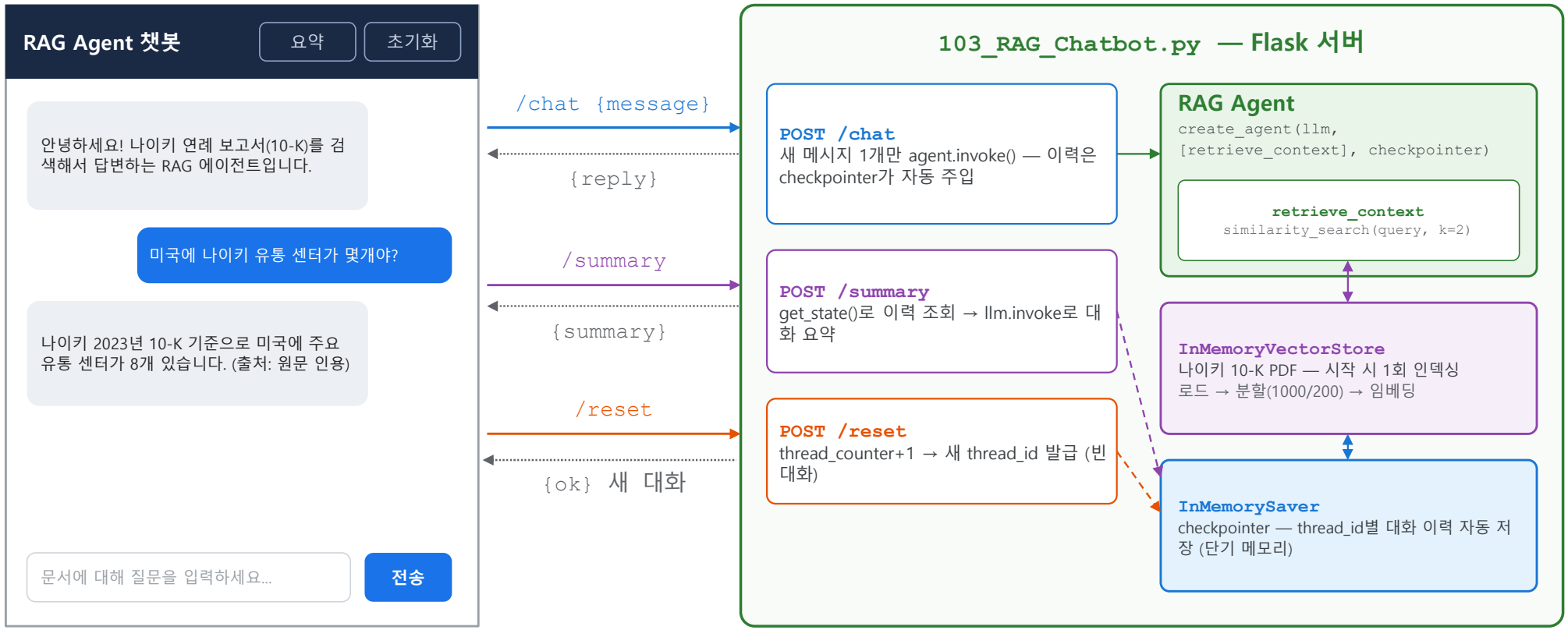
1. **인덱싱 (Indexing)**: 문서를 로드, 분할, 임베딩하여 벡터 스토어에 저장
2. **검색 및 생성 (Retrieval & Generation)**: 사용자 질문에 대해 관련 문서를 검색하고 답변 생성

RAG 패턴 이해

RAG는 두 가지 주요 접근 방식이 있습니다. 이 노트북에서는 **에이전트 기반 RAG를 중심으로** 다루고, 체인 기반 RAG는 마지막에 참고로 비교합니다.

실습 : Flask_WebUI / 103_RAG_Chatbot.py

Hands-on



103_RAG_Chatbot.html
바닐라 JS — fetch로 JSON API 호출

실행: `python 103_RAG_Chatbot.py` → http://localhost:8000

Human-in-the-loop 제어

에이전트 실행 중간에 멈추고(pause) 사람의 승인 · 수정 · 거절을 기다리는 제어 방식

모델이 도구 호출 결정 → ⏸ 실행 직전 멈춤 · 상태 저장 → 사람: approve · edit · reject → Command(resume)로 재개

용도

안전성 향상
잘못된 도구 호출 · 위험한 쿼리를 실행 전에 막는다

품질 향상
사람이 인자를 검토 · 수정(edit)해 결과가 좋아진다

투명성 향상
누가 무엇을 승인 · 거절했는지 남아 감사 · 로그 관리가 쉽다

위험도별 승인 기준

Low	읽기 전용 · 조회	자동 실행
Med	데이터 변경 · 외부 API	수동 승인
High	임의 실행 · 되돌릴 수 없는 작업	수동 승인 + allowed_decisions

활용 시나리오

도구 호출 전
SQL 실행 · API 요청 · 환불 전에 승인

콘텐츠 생성 후
보고서 · 코드를 사람이 검토 · 수정

민감 작업
개인정보 조회 · 재무 거래는 거절(reject)

예) SQL 쿼리 승인

Code

① 사용자 질문
"비싼 트랙 보여줘" 실행

② 모델이 도구 호출
스키마 조회 후
`sql_db_query`로
쿼리 실행 시도

③ ⏸ 실행 직전 멈춤
미들웨어가 가로챔
상태는 체크포인터에 저장
`__interrupt__` 반환

④ 사람이 결정
도구 이름 · 인자를 보고
approve · edit · reject
중 하나 선택

Decision (셋 중 하나)

approve · 승인

모델이 만든 쿼리 그대로 실행
→ **비싼 트랙 5개 조회**

edit · 수정

사람이 고친 쿼리(LIMIT 3)로 실행
→ **3개만 조회**
모델은 원래 쿼리를 다시 실행하지 않음

reject · 거절

도구는 실행 안 함
거절 사유를 모델에 전달
→ **모델이 새 쿼리로 다시 승인 요청**

⑤ 이어 가기 `agent.invoke(Command(resume={"decisions": [decision]}), config)`
같은 thread_id → 멈춘 지점부터 다시 실행 · 거절 후 새 쿼리가 오면 다시 ③으로

코드에서 할 일 3가지

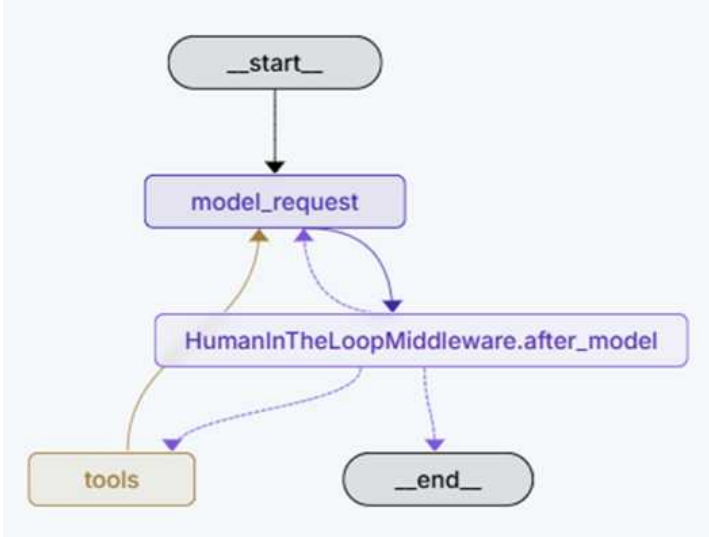
1. **멈출 도구 지정** `HumanInTheLoopMiddleware(interrupt_on={"sql_db_query": {...}})`
2. **멈춘 상태 저장** `checkpointer=InMemorySaver()` ← 없으면 이어 갈 수 없다
3. **묻고 이어 가기** `if result.get("__interrupt__"): agent.invoke(Command(resume=...), config)`

실습 : 104_HITL_Agent.ipynb

Hands-on

실습 목표

- sql_db_query 도구에만 승인 걸기 (HumanInTheLoopMiddleware)
- 승인 · 쿼리 수정 · 거절 세 경우 처리
- 멈춘 실행을 같은 thread_id로 resume해 이어 가기



진행 순서

1. 1~3절: 도구 · interrupt_on · checkpointer 설정
2. 4절: approve / edit / reject로 resume 해 보기
3. 실습 문제 1: 대기 중인 쿼리의 LIMIT을 3으로 edit
4. 실습 문제 2: 개인정보 쿼리 reject → 새 쿼리 approve

DB는 읽기 전용(mode=ro) 연결 → 승인 실수에도 안전
reject하면 sql_db_query가 실행되지 않고
모델이 사유를 반영한 새 쿼리를 만드는지 확인

MCP (Model Context Protocol)

LangChain Tools는 프레임워크 안에서만 — MCP는 표준 규약만 지키면 어떤 클라이언트든 사용

LangChain Tools — Framework 종속

LangChain / LangGraph 앱 내부

- @tool 함수
- Toolkit
- Integrations

같은 파이썬 프로세스 안에서 함수로 호출

MCP — 표준 프로토콜 (프레임워크 독립)

- Claude Desktop
- Cursor
- LangChain 앱
- OpenAI Agents SDK 앱

→ **MCP 서버 1개**
규약대로 도구 노출

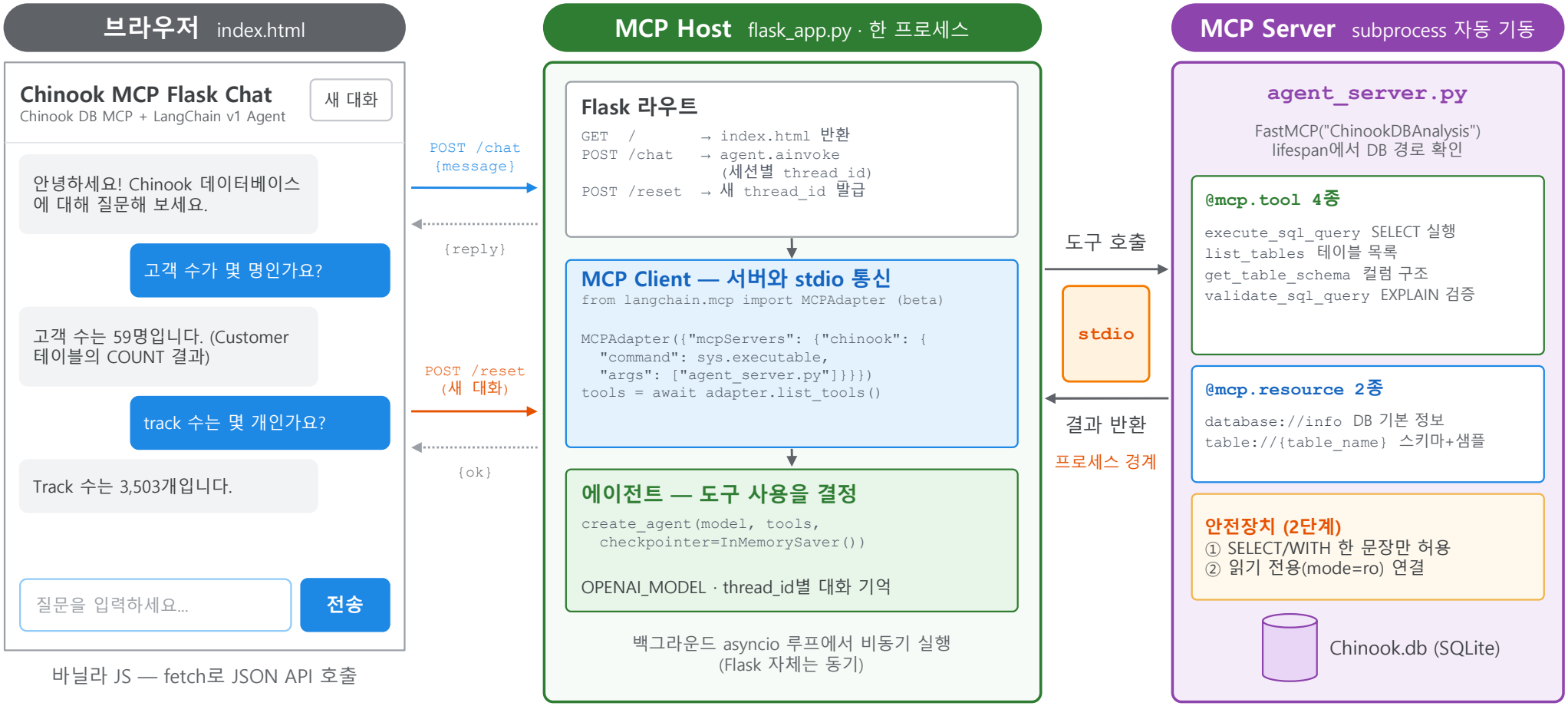
별도 프로세스 · stdio / HTTP로 통신

LangChain에서 쓰기 `from langchain.mcp import MCPAdapter` ← langchain 패키지에 내장 (beta) · 별도 설치 불필요

`await adapter.list_tools()` 가 MCP 서버의 도구를 LangChain 도구로 변환 → create_agent에 그대로 전달

실습 : 105_DB_MCP_Agent — 구성과 동작

Hands-on



실행: `python flask_app.py` → http://localhost:5000

핵심: 에이전트가 결정 → MCP Client가 stdio로 전달 → 서버가 Chinook.db 조회

Resource 테스트 질문

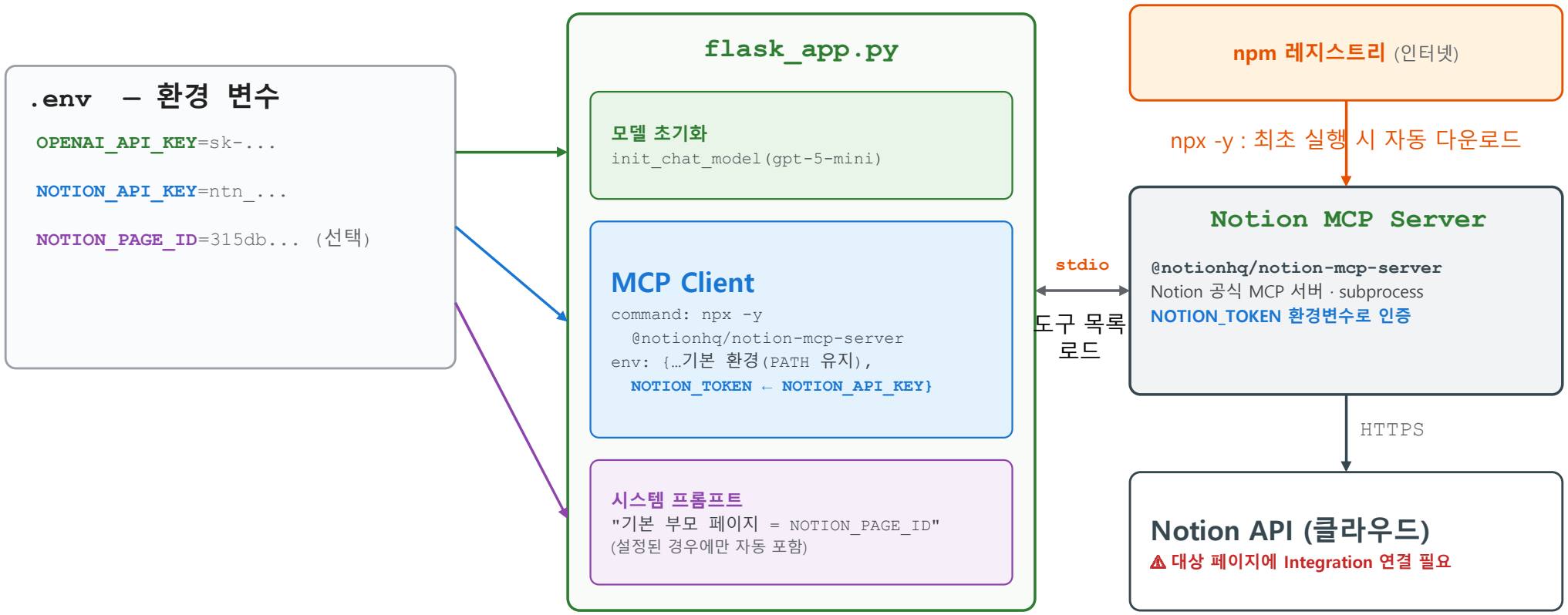
- DB Resource 테스트 질문
  - **1. Database Info Resource:**
    - 데이터베이스 정보를 보여줘"
    - "Chinook 데이터베이스에 대해 알려줘"
    - "데이터베이스 기본 정보를 확인해줘”
  - **2. Table Info Resource:**
    - "Employee 테이블 정보를 보여줘"
    - "Customer 테이블의 구조를 알려줘"
    - "Album 테이블의 스키마를 확인해줘”

- DB Tools 테스트 질문
  - **1. SQL Query Execution Tool:**
    - "직원이 몇 명인가요?"
    - "고객 수를 알려줘"
    - "앨범이 총 몇 개인가요?"
    - "가장 많이 팔린 곡은 무엇인가요?"
    - "매니저는 몇 명인가요?”
  - **2. Table Schema Tool:**
    - "Employee 테이블의 구조를 보여줘"
    - "Customer 테이블의 컬럼 정보를 알려줘"
    - "Track 테이블의 스키마를 확인해줘”
  - **3. List Tables Tool:**
    - "데이터베이스에 어떤 테이블들이 있나요?"
    - "사용 가능한 테이블 목록을 보여줘"
    - "모든 테이블 이름을 알려줘”

- **연속 대화 테스트:**
  - 1단계: "직원이 몇 명인가요?"
  - 2단계: "그 중에서 매니저는 몇 명인가요?”

- **컨텍스트 참조 테스트:**
  - 1단계: "고객이 몇 명인가요?"
  - 2단계: "그 고객들 중에서 미국 고객은 몇 명인가요?"
  - 3단계: "앞서 말한 고객들의 주문 건수는?"

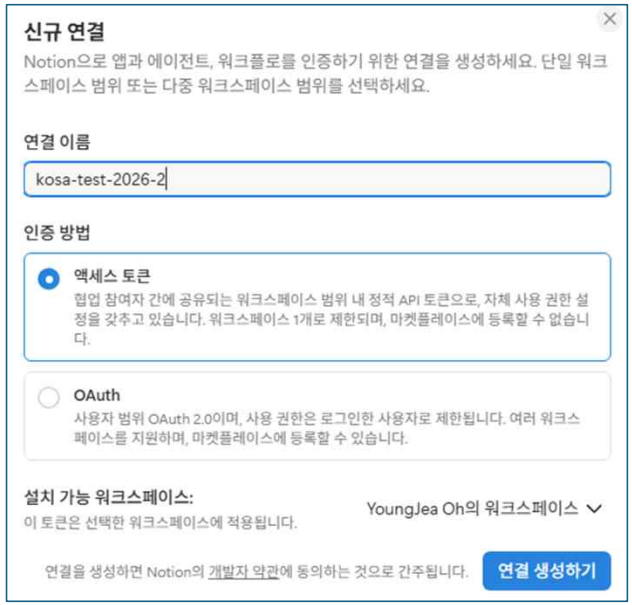
Notion Page 연결 방법



Node.js 미설치 시: https://nodejs.org/ko/download → Windows 설치 프로그램(.msi) · 최초 npx 실행은 패키지 다운로드로 수십 초 걸릴 수 있음 (정상)

Notion API key 생성 - Notion의 개발자 설정(Integrations) 메뉴






```
# .env 파일
OPENAI_API_KEY=

# Notion API 설정
NOTION_API_KEY=
NOTION_PAGE_ID=
```

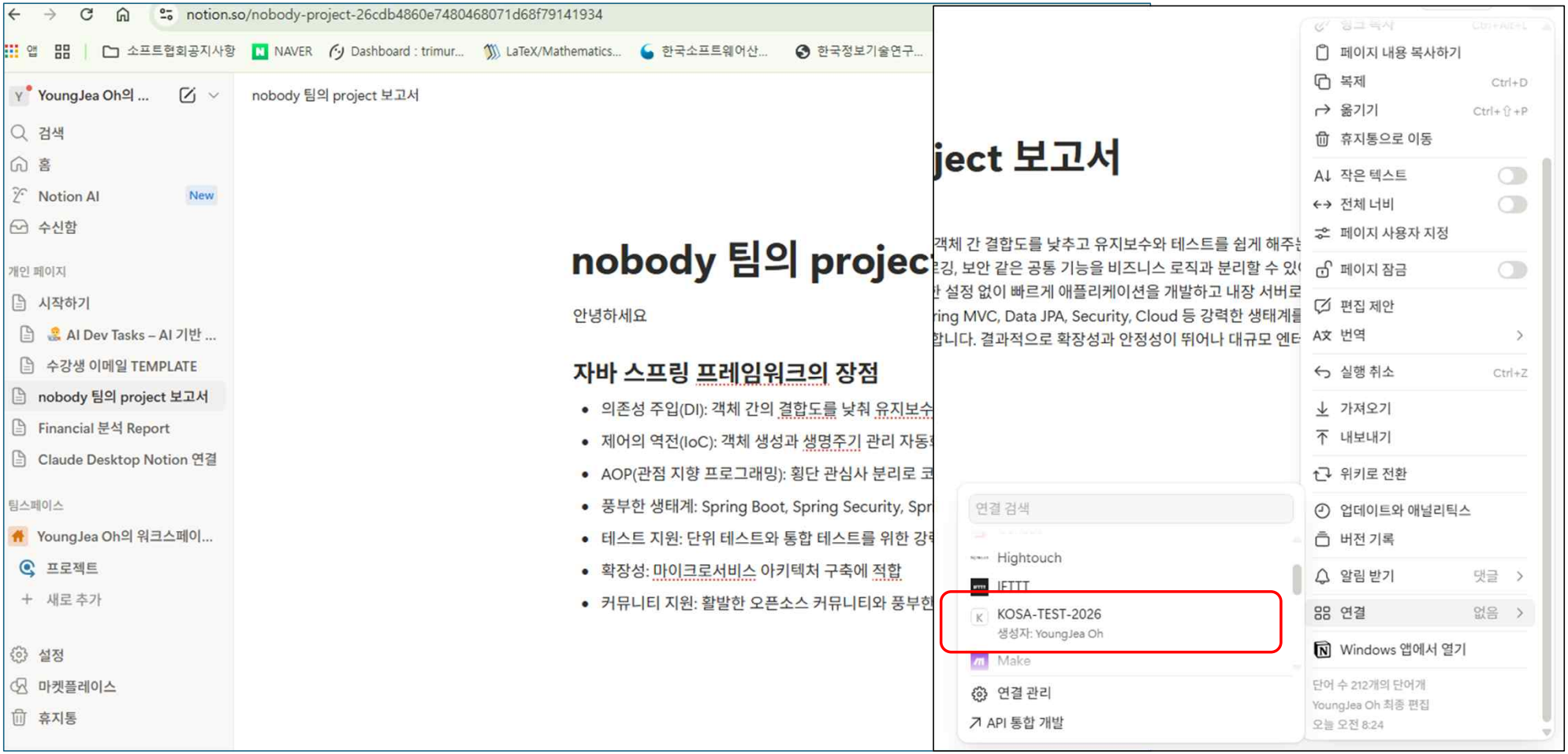
NOTION_API_KEY=ntn_588632**********T0Eyz6xJ

Notion Page 생성

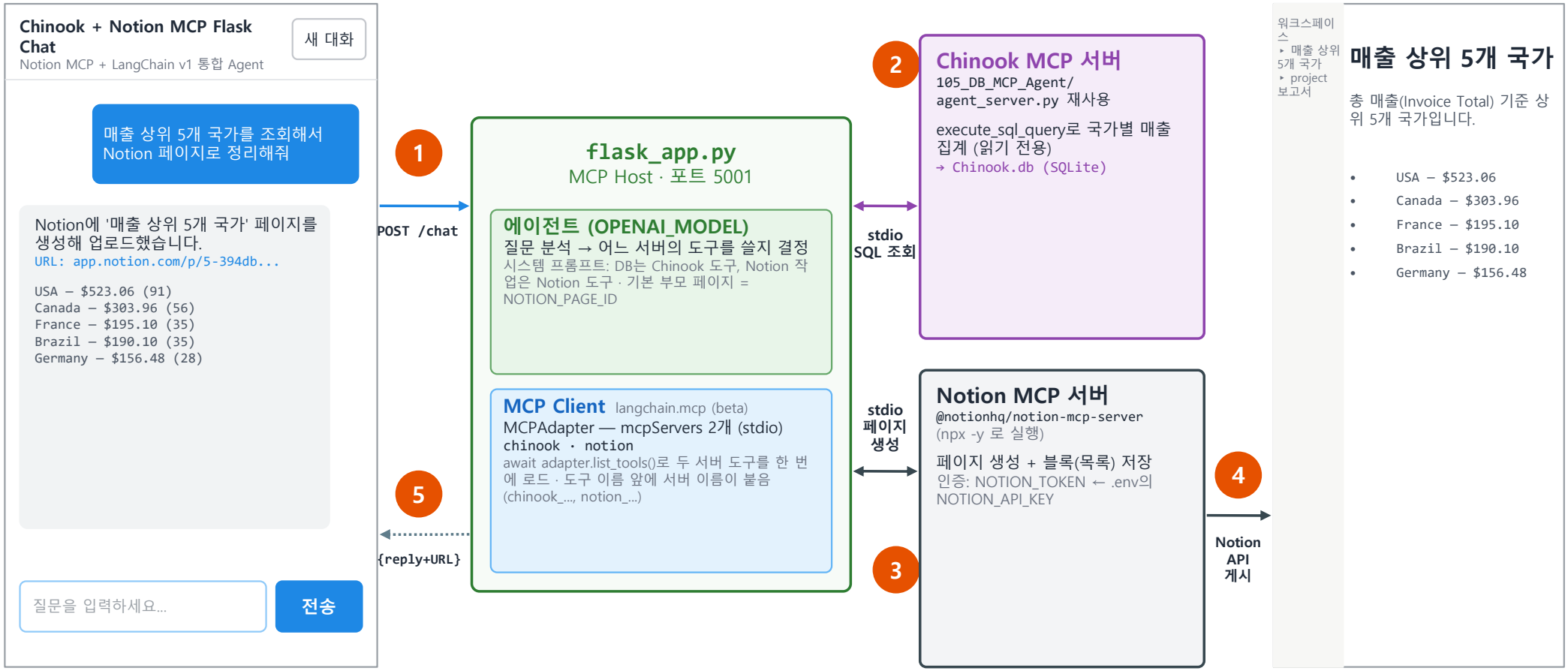
NOTION_PAGE_ID=315db4860e7480eebcbbdd05af92c01e

page-id

- https://www.notion.so/nobody-project-**315db4860e7480eebcbbdd05af92c01e**



실습 : 106_Notion_DB_MCP 구현



Web UI (index.html)

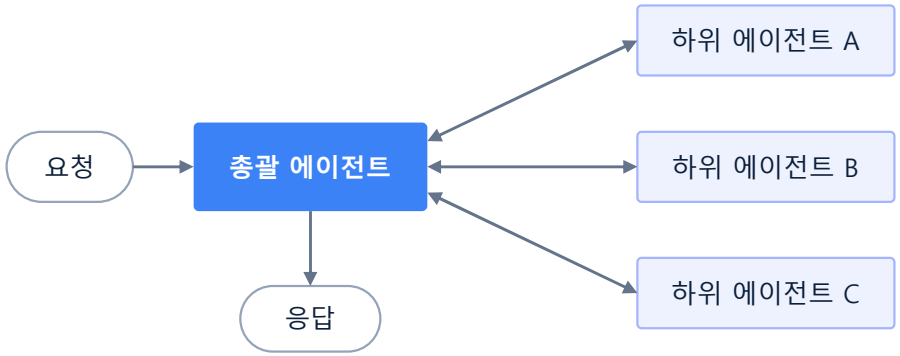
Notion 페이지 (게시 결과)

흐름: ① 질문(POST /chat) → ② Chinook 서버로 SQL 조회 → ③ 결과로 Notion 서버가 페이지 생성 → ④ Notion API 게시 → ⑤ URL 포함 답변 반환

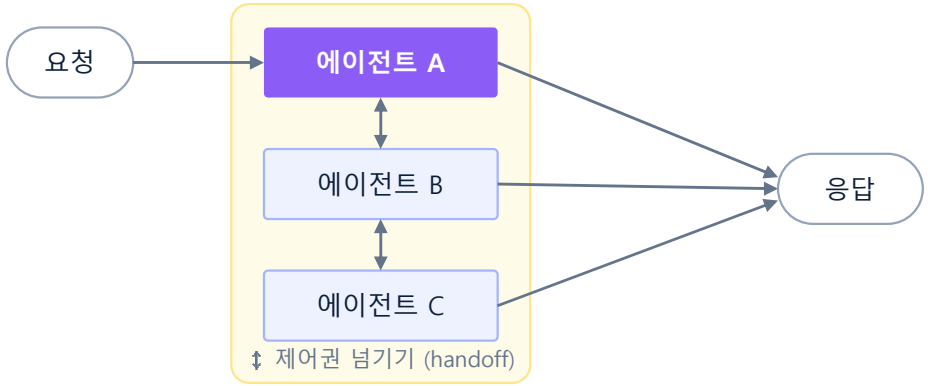
실행: `python flask_app.py` → http://localhost:5001 (105 앱과 동시 실행 가능 · Node.js(npx) 필요 · 대상 Notion 페이지에 Integration 연결 필요)

멀티 에이전트 패턴 흐름도

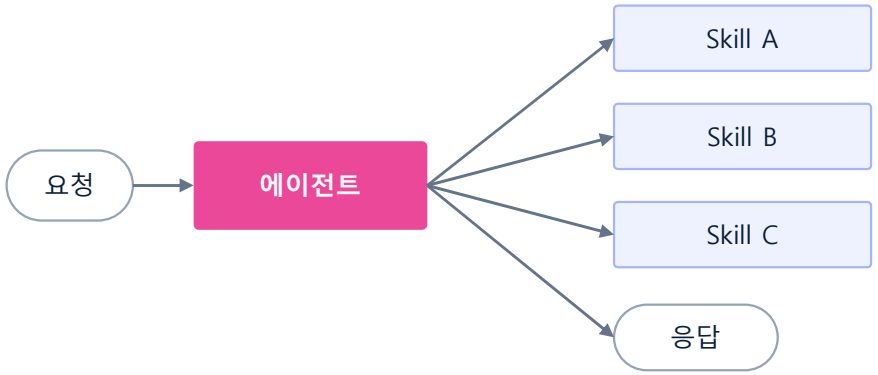
Subagents 총괄 에이전트가 하위 에이전트를 도구로 호출하고 결과를 돌려받는다 (⇄). 모든 흐름이 총괄을 거친다



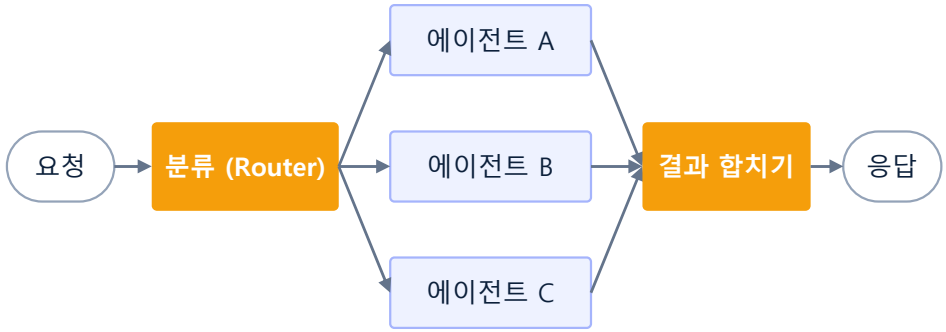
Handoffs 도구 호출로 제어권을 다른 에이전트에 넘긴다. 어느 에이전트든 사용자에게 직접 답할 수 있다



Skills 에이전트 하나가 주도하면서 필요할 때 전문 프롬프트 · 지식을 불러온다



Router 분류 단계가 입력을 하나 이상의 전문 에이전트로 보내고 결과를 합친다



출처: LangChain Docs — Multi-agent (Patterns)

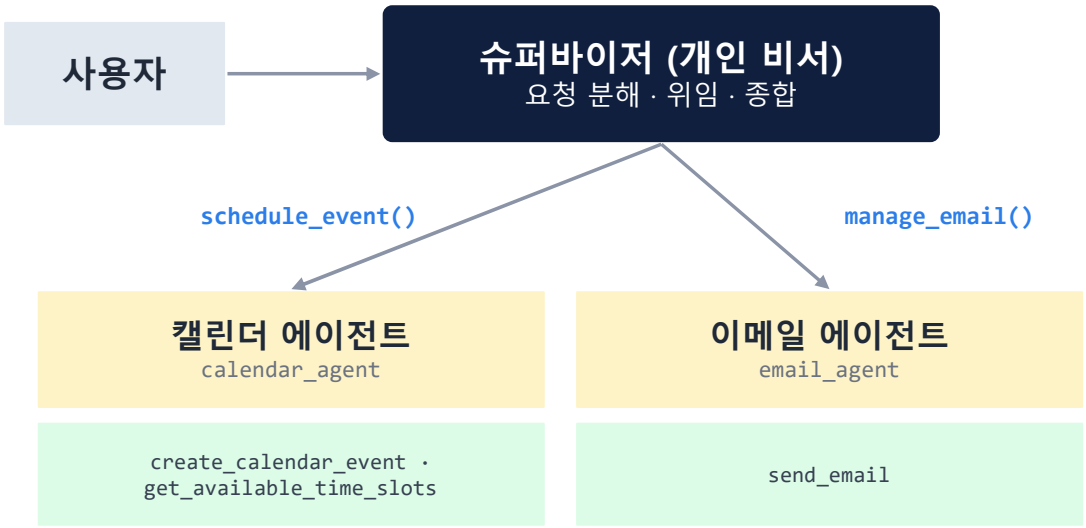
멀티 에이전트 패턴 종합 비교

패턴	동작 방식	팀별 독립 개발	병렬 실행	연속 호출	사용자와 직접 대화
Subagents	총괄 에이전트가 하위 에이전트를 도구로 호출한다. 모든 라우팅은 총괄을 거친다.	★★★★★	★★★★★	★★★★★	★
Handoffs	도구 호출이 상태 값을 바꾸면 에이전트 또는 도구 · 프롬프트가 전환된다.	–	–	★★★★★	★★★★★
Skills	에이전트 하나가 주도하면서 필요할 때 전문 프롬프트 · 지식을 불러온다.	★★★★★	★★★	★★★★★	★★★★★
Router	분류 단계가 입력을 하나 이상의 전문 에이전트로 보내고 결과를 합친다.	★★★	★★★★★	–	★★★
Custom workflow	LangGraph로 정해진 흐름과 에이전트를 섞어 직접 설계한다. 다른 패턴을 노드로 넣을 수 있다.	설계한 흐름에 따라 달라짐 (공식 표에 별점 없음)			

팀별 독립 개발: 팀마다 구성 요소를 따로 유지할 수 있나 · **병렬 실행**: 여러 에이전트를 동시에 실행할 수 있나
연속 호출: 하위 에이전트를 차례로 여러 번 부를 수 있나 · **사용자와 직접 대화**: 하위 에이전트가 사용자와 대화할 수 있나
출처: LangChain Docs — Multi-agent (Patterns · Choosing a pattern)

패턴은 섞어 쓸 수 있다
예: Subagents 총괄이 Router나 Custom workflow를 도구로 부르고, 하위 에이전트는 Skills로 필요한 컨텍스트를 불러온다.

Subagents: 개인 비서 (슈퍼바이저)



하위 에이전트를 @tool로 감싸면 슈퍼바이저의 '도구'가 된다 → 도구 호출로 위임

하위 에이전트 → 슈퍼바이저의 도구로 감싸기

```python
@tool
def schedule_event(request: str) -> str:  # ①
    """자연어 요청으로 일정을 생성/수정"""  # ④
    result = calendar_agent.invoke({  # ②
        "messages": [{"role": "user",
                      "content": request}]
    })
    return result["messages"][-1].text  # ③

supervisor_agent = create_agent(
    model, tools=[schedule_event, manage_email],
    system_prompt=SUPERVISOR_PROMPT)
```

@tool로 감싸기 = 하위 에이전트 실행을 도구 함수 안에 넣는다 → 슈퍼바이저는 다른 도구처럼 호출만 한다

① 입력 = 요청 문장
슈퍼바이저(LLM)가 맡길 일을
request: str 에 직접 써서 호출

② 도구 안에서 에이전트 실행
calendar_agent.invoke()로 하위
에이전트가 자기 도구로 끝까지 처리

③ 최종 답변만 돌려준다
.text로 마지막 메시지만 반환
중간 호출은 슈퍼바이저에 안 남음

④ 함수 이름 · docstring
도구의 name · description이 되어
어느 에이전트에 맡길지 고르는 기준

실습 (310_Subagents_Pattern.ipynb) "다음 주 화요일 오후 2시에 디자인 팀과 1시간 회의를 잡고, 목업 검토 리마인더 메일도 보내줘"
→ 슈퍼바이저가 schedule_event · manage_email 두 도구로 나눠 위임하고 결과를 종합 (단일 요청이면 필요한 에이전트 하나만 호출)

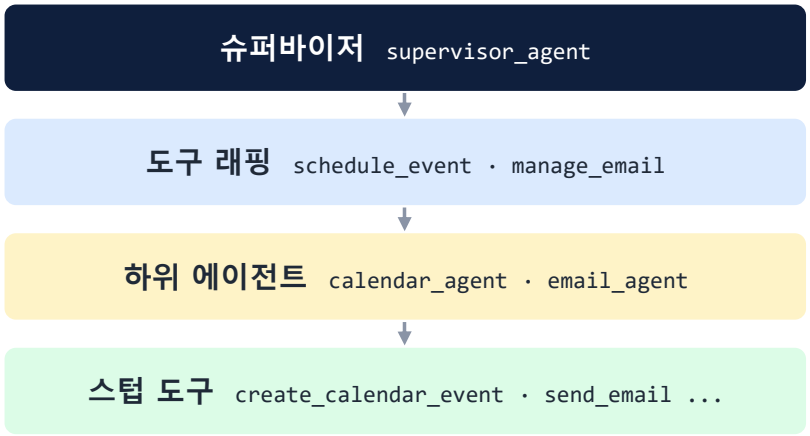
실습 : 310_Subagents_Pattern.ipynb

Hands-on

실습 목표

- 캘린더 · 이메일 하위 에이전트 2개 구성
- 하위 에이전트를 @tool로 감싸 슈퍼바이저의 도구로
- 요청에 맞는 에이전트에만 위임하는지 확인

3계층 구조



진행 순서

1. 1절: 스텁 도구 3개 정의 (캘린더 · 이메일)
2. 2절: calendar_agent · email_agent 생성 · 단독 테스트
3. 3절: schedule_event · manage_email로 래핑
4. 4절: 슈퍼바이저 생성 (SUPERVISOR_PROMPT)
5. 5절: 단일 요청 → 복합 요청 순서로 실행
6. 실습문제: 미니 개인 비서 (날씨 + 메모)

미션: "내일 서울 날씨를 확인하고, 우산을 챙기라는 메모를 남겨줘"
→ check_weather · manage_memo 순서로 위임되는지 확인

Middleware: 안전·비용·품질을 "훅"으로 제어

무엇을 할 수 있나?

- 에이전트 내부에서 일어나는 일을 더 세밀하게 제어
- 실행 루프의 각 단계(모델 호출, 도구 실행, 종료)의 이전(before)과 이후(after)에 개입하는 훅(hook)을 제공
- hook(훅) — 특정 동작이 일어날 때 자동으로 실행되도록 연결된 코드 조각

`before_model`

- 민감정보 차단(PII)
- 도구 호출 제한
- 모델 호출 횟수 제한

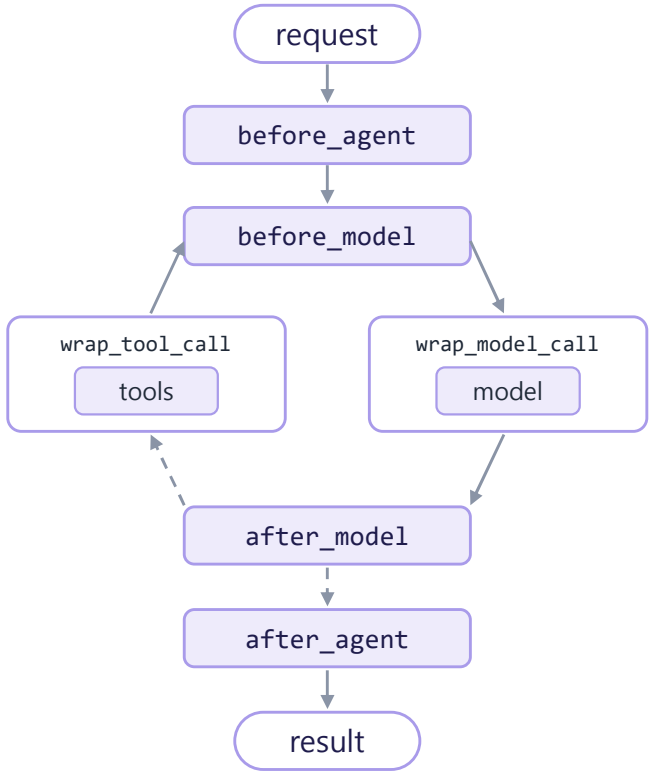
`after_model`

- 응답 검열 / 포맷 검증
- 부적절 출력 제거

`wrap_model_call / wrap_tool_call` → 모델·도구 호출 자체를 감싸 바꾼다

만드는 법: @wrap_model_call 데코레이터 (320) 또는 AgentMiddleware 클래스 (340)

예: 340 SkillMiddleware — 시스템 프롬프트에 스킬 목록 덧붙이기



도구 호출이 있으면 tools → before_model로 다시 돈다

도구에서 실행 정보 읽기 — ToolRuntime · Context

Learn

도구 함수는 두 가지를 받는다

앱 (서버 코드)
`agent.invoke(입력, context=...)`

모델 (LLM)
도구 스키마를 보고 인자를 채움

② runtime · 앱이 넣음
`runtime.context` (고정값)
`runtime.state` (대화·단계)
`runtime.tool_call_id`
(결과 메시지 연결)

① 인자 · 모델이 채움
`status="in_warranty"`

고객 ID를 인자로 두면?
모델이 엉뚱한 값을 넣을 수도

③ 도구 함수에서 만남
`def record_warranty_status(status, runtime: ToolRuntime)`

→ 고객 ID처럼 **바뀌면 안 되는 값**은 인자가 아니라 **Context**로
runtime 매개변수는 도구 스키마에서 빠져 모델에게 보이지 않는다

320 노트북 턴 2에서 일어나는 일

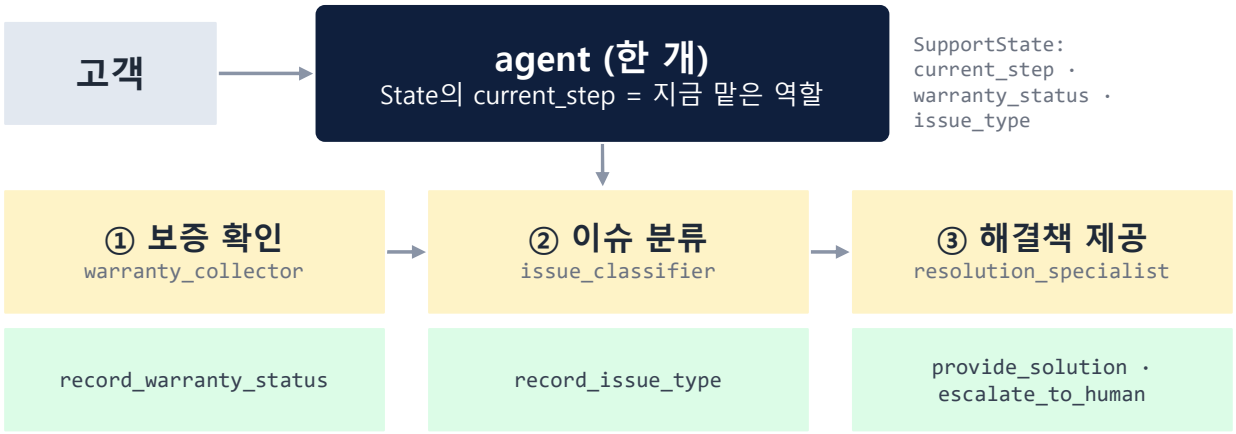
도구가 인자 + runtime을 받아 State를 바꾸는 순간

1. **사용자가 답한다**
   "네, 아직 보증 기간 내입니다."
2. **모델이 도구를 고르고 인자를 채운다 (①)**
   `record_warranty_status(status="in_warranty")`
3. **도구 실행 — runtime은 자동으로 들어온다 (②)**
   `runtime.tool_call_id` → ToolMessage를 이 호출에 연결
4. **Command(update)로 State를 바꾼다**
   `warranty_status = "in_warranty"`
   `current_step = "issue_classifier"`

→ 다음 모델 호출부터 '이슈 분류' 단계의 프롬프트·도구가 적용된다

- **ToolRuntime[Context, State]:** 320은 Context 없이(None) State 타입만 지정
- **`record_issue_type`**도 같은 방식 — issue_type 기록 후 current_step을 resolution_specialist로

Handoffs: 한 에이전트가 단계별로 역할 바꾸기



기록 도구가 돌려준 Command로 current_step이 바뀌고, 다음 모델 호출 때 미들웨어가 그 단계의 프롬프트·도구로 교체한다

모델을 부를 때마다 미들웨어가 하는 일

`apply_step_config` (`@wrap_model_call`)

① **현재 단계 읽기**
`current_step` — 없으면 warranty_collector

② **진입 조건 확인**
`requires` 값이 State에 없으면 오류

③ **프롬프트 · 도구 교체**
`STEP_CONFIG`에서 꺼내 `request.override`

④ **모델 호출 → 기록 도구가 단계 변경**
다음 호출에서 ①부터 다시

Handoffs = 에이전트를 여러 개 만들지 않고, 한 에이전트가 State에 따라 프롬프트·도구를 바꿔 역할을 넘겨받는다

노트북 7절 — 같은 thread_id로 4턴 대화 체크포인터가 턴 사이 State를 이어 줌 · 8절 보증 만료면 마지막에 escalate_to_human · 도구 호출 시점은 모델에 따라 한 턴 달라질 수 있음

턴	사용자	단계 (current_step)	호출 도구 → State 변화
1	"안녕하세요, 제 휴대폰 화면이 깨졌어요."	`warranty_collector`	보증 기간 내인지 묻기만 함 (도구 없음)
2	"네, 아직 보증 기간 내입니다."	`warranty_collector → issue_classifier`	`record_warranty_status → warranty_status = in_warranty`
3	"화면이 물리적으로 깨졌어요. 떨어뜨려서..."	`issue_classifier → resolution_specialist`	`record_issue_type → issue_type = hardware`
4	"어떻게 해야 하나요?"	`resolution_specialist`	`provide_solution` (보증 수리 안내)

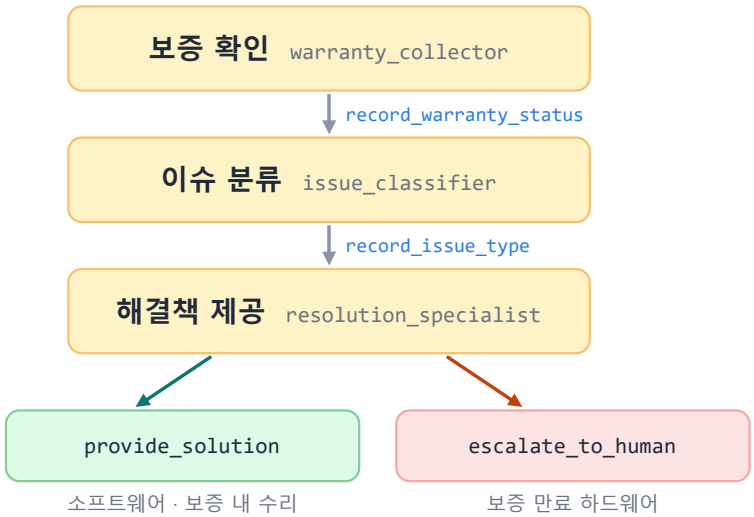
실습 : 320_Handoffs_Pattern.ipynb

Hands-on

실습 목표

- current_step으로 한 에이전트가 세 역할을 순서대로 수행
- 기록 도구가 Command(update)로 State를 바꿔 다음 단계로
- 미들웨어가 단계별 프롬프트·도구를 교체

단계 흐름 (current_step)

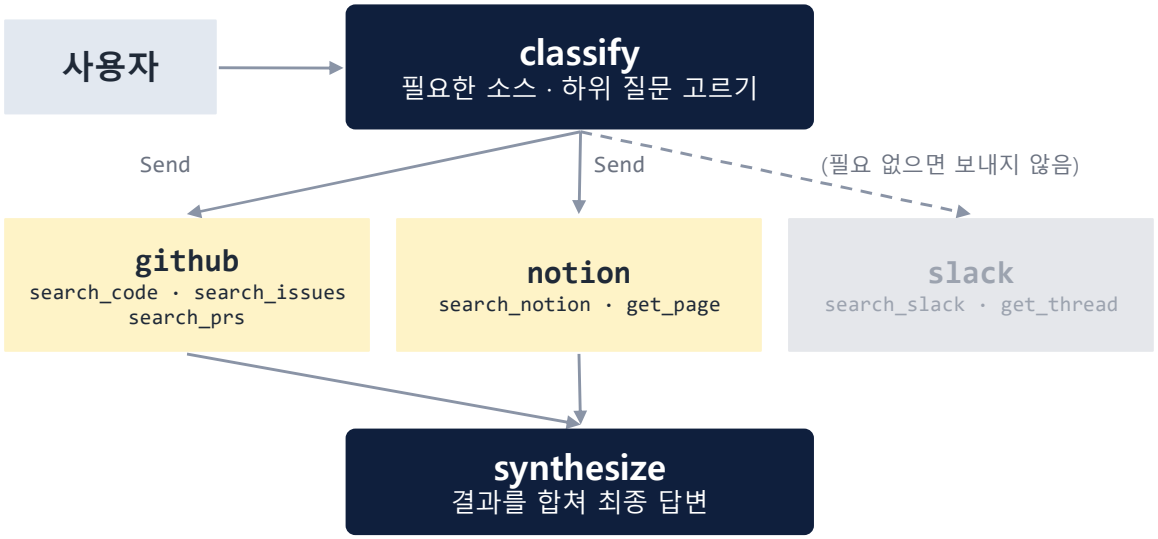


진행 순서

1. 1절: SupportState (current_step 등 3개 필드)
2. 2절: 기록 도구 — Command(update=...)
3. 3~4절: 단계별 프롬프트 · STEP_CONFIG · 미들웨어
4. 5절: create_agent(state_schema, middleware)
5. 7~8절: 같은 thread_id로 4턴 · 보증 만료 시나리오
6. 실습문제: 보증 기간 내 + 소프트웨어 문제

미션: "휴대폰 앱이 자꾸 강제 종료돼요"
→ 보증 내 → 소프트웨어 → 해결책 요청
매 턴 current_step 확인 · 마지막에 provide_solution 호출 확인

Router: 분류 → 동시 실행 → 합치기



분류 단계가 필요한 전문 에이전트에게만 동시에 보내고, 결과를 하나로 합친다

핵심 구현 포인트

- **classify:** with_structured_output으로 { source, query } 목록을 받는다
- **route_to_agents:** Send(노드, 입력)를 여러 개 돌려주면 전문 에이전트가 동시에 실행
- **results:** operator.add 리듀서로 모은다 → 동시에 와도 덮어쓰지 않는다
- **synthesize:** 결과를 합쳐 final_answer 하나로

실습 (330_Router_Pattern.ipynb) — 사내 지식 소스 3곳을 묶는 라우터

- 질문: "API 요청을 어떻게 인증하나요?"
- classify → github(코드 구현) · notion(내부 가이드)에 동시에 Send → synthesize로 답변 종합 (slack은 보내지 않음)
- "코드 리뷰 프로세스는 어떻게 되나요?" → notion · slack만 선택 — 질문마다 고르는 소스가 다르다

LangGraph 기초 — State · Node · Edge

Concept



State RouterState

query
classifications
results ← 이어 붙임
final_answer

모든 노드가 오른쪽 State를 읽고, 바꿀 필드만 돌려준다

State · Reducer

노드들이 함께 쓰는 공용 데이터
리듀서가 붙은 필드는
덮어쓰지 않고 이어 붙인다

```
Annotated[list, operator.add]
```

Node

state를 받아 바꿀 필드만
dict로 반환하는 함수
안에서 LLM · 에이전트 호출 가능

```
return {"classifications": [...]}
```

Edge

다음 노드를 **고정으로 연결**
여러 노드가 한 곳에 모이면
모두 끝난 뒤 실행

```
add_edge("github", "synthesize")
```

Conditional edge · Send

함수의 반환값으로 다음 노드를 고른다
Send를 여러 개 돌려주면 **그 노드들이 동시에 실행**

```
Send(c["source"], {"query": c["query"]})
```

START · END · compile

START에서 시작해 END에서 끝난다
compile() 하면 invoke · stream 가능한 그래프가 된다

```
graph = workflow.compile();  graph.invoke({...})
```

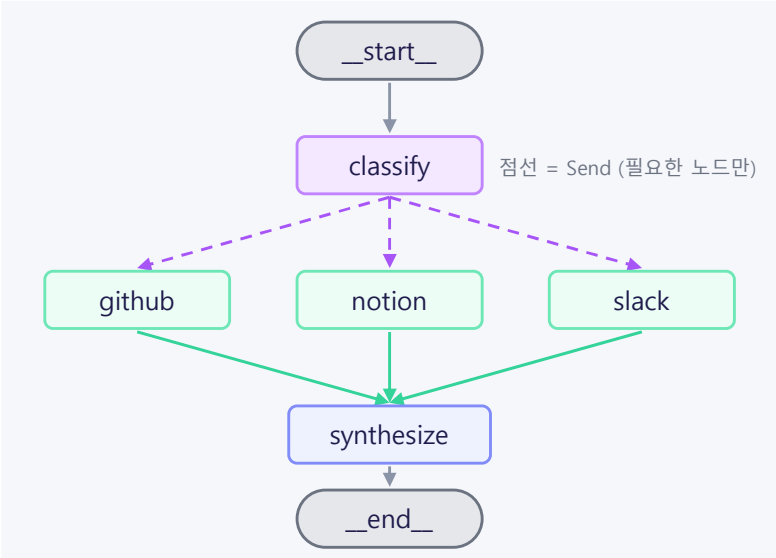
create_agent도 내부적으로 model ↔ tools 반복 그래프 — 흐름을 **직접 설계**할 때(330 Router)는 StateGraph로 그린다

실습 : 330_Router_Pattern.ipynb

Hands-on

실습 목표

- 질문을 분류해 필요한 소스와 하위 질문 고르기
- Send로 여러 전문 에이전트를 동시에 실행
- operator.add로 결과를 모아 최종 답변 하나로

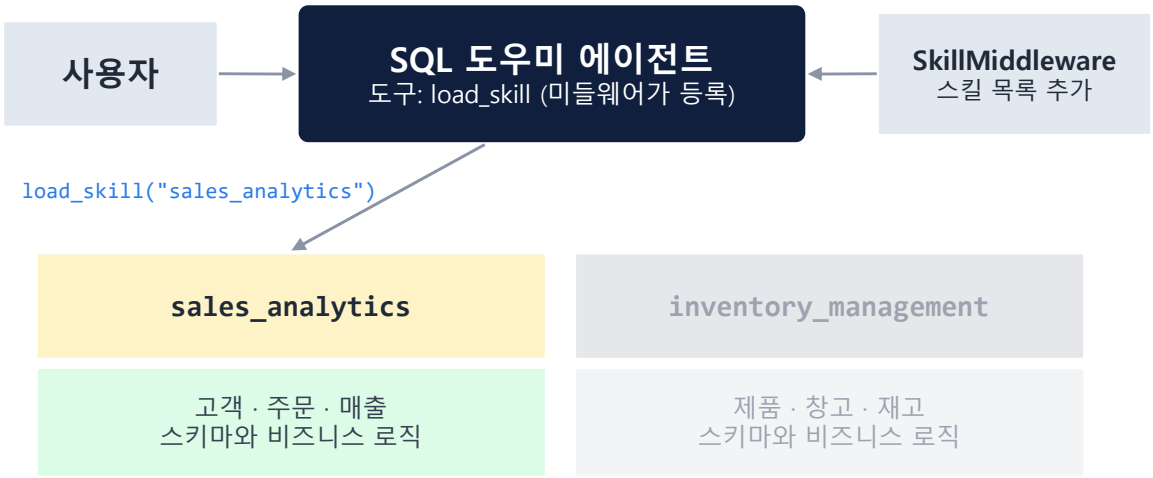


진행 순서

1. 1절: RouterState (results에 operator.add)
2. 2~3절: 소스별 도구 + 전문 에이전트 3개
3. 4절: classify_query (with_structured_output)
4. 5절: route_to_agents (Send) + 에이전트 노드
5. 6~7절: synthesize_answer + StateGraph 조립
6. 8~10절: 실행 · 스트리밍 · 쿼리별 소스 확인

미션: "API 요청을 어떻게 인증하나요?" → github · notion
"코드 리뷰 프로세스는?" → notion · slack
질문마다 Send되는 소스가 달라지는지 확인

Skills: 필요한 스킬만 불러오는 SQL 도우미



필요한 스킬만 load_skill로 통째로 불러온다 (노란색 = 불러온 스킬)

스킬 하나 = description은 프롬프트에, content는 필요할 때

	description · 시스템 프롬프트	content · load_skill 결과
들어가는 때	매 모델 호출 (미들웨어)	모델이 필요하다고 판단할 때
내용	스킬 이름 + 한 줄 설명	테이블 스키마 · 규칙 · 예시 쿼리
크기	짧다 (스킬 수만큼 한 줄)	길다 → 필요한 것만 로드

340 노트북 흐름 — 1절부터 5절까지

스킬 정의 → 불러오는 도구 → 목록 알리는 미들웨어 → 에이전트 → 실행

1. **1절 · 스킬 정의 (Skill TypedDict)**
   `name` · `description` · `content` — SKILLS 2개
2. **2절 · load_skill 도구**
   `skill_name`으로 찾아 content 반환 · 없으면 사용 가능한 스킬 목록 안내
3. **3절 · SkillMiddleware**
   `tools = [load_skill]` 등록 + 모델 호출마다 시스템 프롬프트에 "## 사용 가능한 스킬"(설명만) 추가
4. **4절 · 에이전트 생성**
   `create_agent(..., middleware=[SkillMiddleware()])` — tools 인자 없이도 load_skill 사용 가능
5. **5절 · 실행**
   SQL 질문 → `load_skill("sales_analytics")` → 스키마 · 매출 규칙(status='completed') 반영 SQL

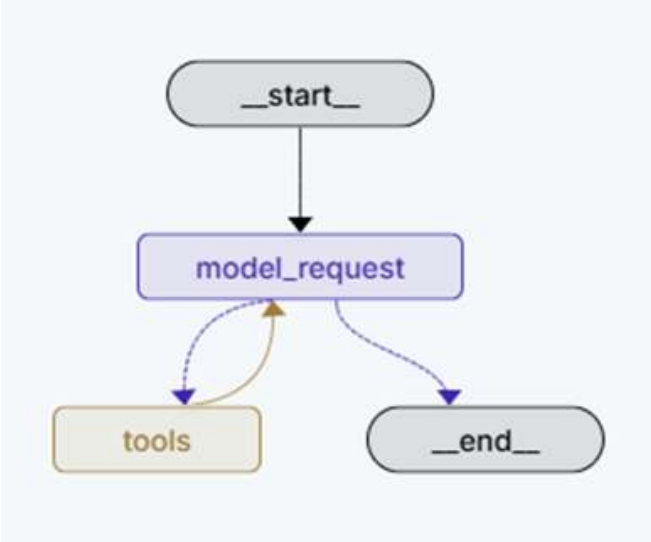
실습 (340_Skills_Pattern.ipynb) "지난달 매출 1000달러 이상인 주문을 한 고객 목록을 조회하는 SQL을 작성해줘"
→ sales_analytics만 로드 (inventory_management는 안 불러옴) · 트레이드오프: 도구 호출 1번만큼 지연, 로드를 강제하지는 못함

실습 : 340_Skills_Pattern.ipynb

Hands-on

실습 목표

- 스킬을 name · description · content로 나눠 정의
- 프롬프트에는 설명만, 본문은 load_skill로 필요할 때만
- 질문마다 어떤 스킬을 불러왔는지 확인



진행 순서

1. 1절: Skill(name · description · content) 2개
2. 2절: load_skill 도구 — 이름으로 content 반환
3. 3절: SkillMiddleware (wrap_model_call)
4. 4절: create_agent(middleware=[SkillMiddleware()])
5. 5절: SQL 질문 → load_skill 호출 확인
6. 실습문제: 사내 규정 스킬 (휴가 · 경비)

미션: "연차 휴가는 며칠인가요?"
→ vacation_policy 스킬만 로드되는지 확인
(expense_policy는 불러오지 않아야 함)

감사합니다.